# Radar Tracking Software Pipeline Optimisation for UAV Detect-and-Avoid

NOAH WEST

B.Eng (Hons)


Supervisor: Dr. Donald Dansereau

A thesis submitted in fulfilment of
the requirements for the degree of
Bachelor of Engineering (Honours)

School of Aerospace Mechanical and Mechatronics Engineering
Faculty of Engineering
The University of Sydney
Australia

25 May 2026

# Abstract

This progress report presents the design, implementation, and partial evaluation of a software-defined radar signal processing pipeline for unmanned aerial vehicle (UAV) detect-and-avoid (DAA) applications. The pipeline is designed for a small monopulse radar system based on four Texas Instruments AWR2243 radar chips, targeting execution on an NVIDIA Jetson Xavier NX embedded GPU platform. The preprocessing subsystem, comprising windowing, FFT, and coherent integration stages, and the detection subsystem, comprising CFAR detection and monopulse angle sensing, have been implemented and evaluated on both synthetic and real radar hardware data. Results demonstrate that the Hamming window is appropriate for radar applications requiring close-target resolution, that coherent integration improves SNR in accordance with theoretical predictions, that the numerical method for determining the CFAR threshold coefficient is accurate across a range of false alarm rates, and that a GPU-based CFAR implementation is most appropriate for the Xavier platform across all practical window sizes. The ablation study of the detection subsystem confirms that CFAR dominates subsystem latency, identifying it as the primary target for further optimisation. The multi-target tracking stage, including clustering, data association, and the tracking filter, remains to be implemented and evaluated. An updated project plan is provided, outlining the methodology, timeline, and risk assessment for the remaining work.

# Statement of Contributions

I, **Noah West**, declare that the intellectual content of this thesis is the product of my own work and that all assistance received in the preparation of this thesis and relevant sources have been appropriately acknowledged. The contributions I made to this research include:

- Undertaking the background research and completing the literature review.
- Defining the research scope and developing the problem statement in consultation with my supervisor and the industry partner.
- Designing the methodology, experimental approach, and test plan with guidance from my supervisor.
- Designing and implementing, in collaboration with engineers from the industry partner *Mission Systems*, the software pipeline described in this work.
- Profiling and evaluating the pipeline software to produce the reported results.
- Capturing radar data for use in testing and validation.
- Conducting the experiments, including configuration of the test setup and execution of data collection procedures.
- Performing analysis of results, including quantitative evaluation using appropriate metrics to validate conclusions.
- Interpreting findings, and writing the discussion and conclusions, incorporating feedback from my supervisor and relevant stakeholders.
- Writing the thesis and preparing figures, tables, and supporting material.

The above represents an accurate summary of my contribution.


**Noah West**
Honours Candidate
School of Aerospace, Mechanical and
Mechatronic Engineering,
The University of Sydney
Date: 25th May 2026

**Dr. Donald Dansereau**
Senior Lecturer
Australian Centre for Field Robotics,
The University of Sydney
Date: .....

# Contents

# Acronyms

**ADC:** Analog-to-Digital Converter

**ARM:** Advanced RISC Machine

**ASIC:** Application-Specific Integrated Circuit

**BVLOS:** Beyond Visual Line of Sight

**CAGR:** Compound Annual Growth Rate

**CASA:** Civil Aviation Safety Authority

**CFAR:** Constant False Alarm Rate

**CPU:** Central Processing Unit

**CSI:** Camera Serial Interface

**CUDA:** Compute Unified Device Architecture

**DAA:** Detect-and-Avoid

**DAC:** Digital-to-Analog Converter

**DBSCAN:** Density-Based Spatial Clustering of Applications with Noise

**DFT:** Discrete Fourier Transform

**DRAM:** Dynamic Random-Access Memory

**DSP:** Digital Signal Processing

**DTFT:** Discrete-Time Fourier Transform

**EASA:** European Union Aviation Safety Agency

**EKF:** Extended Kalman Filter

**FFT:** Fast Fourier Transform

**FMCW:** Frequency-Modulated Continuous Wave

**FPGA:** Field-Programmable Gate Array

**GNN:** Global Nearest Neighbour

**GPU:** Graphics Processing Unit

**IFR:** Instrument Flight Rules

**IMM:** Interacting Multiple Model

**JPDA:** Joint Probabilistic Data Association

**LFM:** Linear Frequency Modulation

**LFMCW:** Linear Frequency-Modulated Continuous Wave

**MHT:** Multiple Hypothesis Tracking

**MOT:** Multiple Object Tracking

**MSE:** Mean Squared Error

**MTT:** Multi-Target Tracking

**NEES:** Normalised Estimation Error Squared

**NIS:** Normalised Innovation Squared

**NN:** Nearest Neighbour

**PCB:** Printed Circuit Board

**PCIe:** Peripheral Component Interconnect Express

**PDA:** Probabilistic Data Association

**PLL:** Phase-Locked Loop

**PVA:** Position-Velocity-Acceleration

**SIMD:** Single Instruction Multiple Data

**SIMT:** Single Instruction Multiple Threads

**SNR:** Signal-to-Noise Ratio

**SoC:** System-on-Chip

**SWaP:** Size, Weight, and Power

**TCPA:** Time to Closest Point of Approach

**TOPS:** Tera Operations Per Second

**UAS:** Unmanned Aircraft System

**UAV:** Unmanned Aerial Vehicle

**UKF:** Unscented Kalman Filter

**UTM:** Unmanned Traffic Management

**VFR:** Visual Flight Rules

# List of Figures

# List of Tables

CHAPTER 1

# Introduction

---

With more Unmanned Aerial Vehicle (UAV)s in the air each year, and a growing desire for them to operate in controlled airspaces, there is a concern regarding their ability to maintain separation from hazards in the air. United States and European federal agencies are developing policy that would require detect-and-avoid (DAA) systems for UAVs, which parallel the visual and instrument flight rules (VFR & IFR) that dictate how human pilots avoid traffic [1] [2] [3].

While some proposed DAA systems utilise optical and infrared sensors [3], radar sensors may also be suitable, as they can perform sensing over great distances in all weather conditions, due to their relatively low atmospheric attenuation [4]. The inconvenient side effect of their long wavelength property is that angular resolution is much lower compared to similarly sized optical systems. If the challenges stemming from this limitation are overcome, the implementation of radar as the primary sensing mode or an additional sensing mode will improve the perception capabilities, and thus the safety, of UAVs.

Historically, airborne radar has been used for large aircraft, where size weight and power (SWaP) constraints are not strict. The size and weight of radar systems has been the largest barrier to their use on smaller platforms, [5], [6], [7]. This historical constraint is now weakening because both sides of the radar stack have miniaturised. Texas Instrument's AWR2243 is a single 76-81 GHz radar chip that integrates phase-locked loops (PLLs), three transmitters, four receivers, a mixer, an ADC, and self calibration, all on a 10.4mm $\times$ 10.4mm printed circuit board (PCB) -friendly package. Higher frequency, lower noise, electronics and the miniaturisation of the transistor have been key enablers of this technology. Higher frequency radar allows for higher resolution imaging or smaller aperture; lower noise allows

for increased sensitivity to targets. On the compute side, modern UAV systems often have a dedicated, high compute companion computer in addition to the flight compute. NVIDIA's Jetson series computers such as Xavier NX, Orin Nano and Orin NX provide 21, 67, and 157 trillion operations per second (TOPS) on 8-bit integers respectively, while each weigh under 50g (without a heat sink).

Plextek's mmWave-based 60 GHz sense-and-avoid radar for UAVs demonstrates the feasibility of pairing miniaturised radar hardware with modern edge computing for the detection of people, vehicles, poles, and buildings at ranges of hundreds of metres. Echodyne's EchoFlight, a 817 g commercial radar systems for UAVs that can detect small drone targets at a range of 750 m, reflects the onset of the technology into industry [8]. The wider UAV market shows a similar shift toward more computationally demanding onboard tasks. Skydio's X10, for instance, incorporates both an NVIDIA Jetson Orin SoC and a Qualcomm QRB5165 SoC within a 2.11 kg platform while retaining up to 40 minutes of flight time, illustrating that substantial embedded compute is now compatible with practical UAV SWaP limits. These developments suggest that recent improvements in radar miniaturisation and embedded heterogeneous computing are promoting a new class of computation-heavy UAV sensing systems, making it timely to revisit radar target tracking and to determine the most suitable signal-processing pipeline for UAV detect-and-avoid applications.

Despite the challenges, UAVs are the fastest growing market for radar systems, projected to grow with a compound annual growth rate (CAGR) of 11.22% globally through 2030 [9]. To make these emerging radar systems more performant, accessible, and safe, it would be valuable to find the most suitable radar signal processing pipeline for UAV DAA systems, revisiting the old problem of radar target tracking in the face of unique hardware conditions.

Software-defined architecture (SDAs) are increasingly preferred over hardware defined radar systems for UAV applications as software modifications can be shipped faster than hardware modifications [9]. As a result, graphics processing units (GPUs) are favoured over non-programmable digital signal processors such as application specific integrated circuits (ASICs). GPU-accelerated implementations are also favourable for computationally intensive stages of the radar processing chain, as many core operations, such as fast Fourier transforms (FFTs)

and matrix operations, are highly data-parallel, allowing for large increases in throughput when compared to serial processors. Additionally, GPU-based pipelines can leverage mature developer friendly, GPU software ecosystems and optimised libraries [10]. Additionally, the incorporation of AI into radar tracking algorithms has further encouraged these highly parallel, GPU-based pipelines [11]. Key open questions remain around balancing the compute saved through GPU parallelism against the cost of data movement and synchronisation, and how unified (shared) CPU–GPU memory on embedded platforms changes this trade-off.

Given the miniaturisation of radar enabling technology, the potential benefits for radar base DAA systems, and the range limitations of systems suitable for small radar systems. This report proposes a software pipeline design that allows for the radar tracking of aerial hazards on UAVs with strict SWaP constraints. Particular emphasis will be placed on maximising the sensitivity to faraway targets, a requirement for operating in shared airspace, and minimising computation resources and latency.

Existing research demonstrates that compact radar hardware is now sufficiently mature for UAV DAA applications. SWaP-compliant systems have been built, detection at DAA-relevant ranges has been achieved, and processing feasibility on constrained embedded platforms has been established. However, three important gaps are identified.

First, existing hardware demonstrations validate detection only against large, high radar cross-section targets such as buildings and fixed-wing aircraft, leaving the practical sensitivity floor of compact UAV radar systems against small moving airborne targets unknown.

Second, no work has benchmarked a GPU-based radar pipeline in a UAV DAA setting. Existing studies are limited to FPGA and ARM CPU implementations, evaluate single algorithm sequences on single hardware configurations, and do not profile architectural bottlenecks such as host–device data transfer and thread synchronisation overhead.

Thirdly, angle sensing algorithms for low-SNR monopulse radar have only been validated under idealised white Gaussian noise with single simulated targets; no studies integrate these algorithms into a complete end-to-end range–Doppler tracking pipeline for UAV DAA.

The core problem this thesis addresses is the absence of a principled, hardware-aware understanding of how algorithm selection and implementation choices jointly affect the sensitivity and computational performance of compact monopulse radar systems for UAV DAA. This thesis constructs that understanding through a systems-level investigation of the software design space of a GPU-based radar tracking pipeline, with specific contributions as follows:

- A comparison of the algorithms necessary for monopulse radar, assessed by detection sensitivity and track quality in multi-target aerial scenarios, to inform algorithm selection.
- A comparison of alternative low-level implementations of the selected algorithms, quantifying their effects on end-to-end throughput and latency in highly parallelised GPU pipelines, to inform implementation choices.
- An assessment of how these algorithm and implementation conclusions vary across embedded hardware configurations, providing hardware-aware design recommendations.

This thesis concerns the software design and evaluation of radar tracking pipelines for UAV DAA under SWaP constraints. The scope covers processing stages from raw radar samples through windowing, FFT, coherent integration, CFAR detection, monopulse angle estimation, data association, and tracking filters, on embedded GPU platforms suitable for small and medium UAVs.

Radar hardware design, full-scale field trials, FPGA and ASIC implementations, and comparisons with optical or infrared sensing are outside scope. The thesis does not claim certification readiness; it provides comparative evidence and design guidance for onboard radar processing under realistic embedded constraints.

The radar sensor is assumed to be a small monopulse radar whose aperture and power budget are representative of a UAV-compatible installation. The target compute platform is GPU-enabled embedded hardware with shared or unified CPU/GPU memory, as found in many embedded systems-on-module, though CPU-only and discrete-memory configurations are included where needed to clarify architectural trade-offs.

By benchmarking alternative implementations of radar signal processing stages and comparing tracking algorithms, this work quantifies how these choices affect detection sensitivity, false alarm rate, and computational performance. In addition to characterising the algorithms themselves, the project produces a reproducible evaluation framework — including computational profiling, ablation studies, and parameter sweeps — that can be applied to comparable radar research. The resulting design recommendations provide an evidence-based guide for engineers developing small radar systems, and support the integration of UAVs into civil airspace where reliable onboard tracking of non-cooperative targets is essential.

More broadly, improved DAA capability supports the safe scaling of beyond-visual-line-of-sight UAV operations in shared airspace, enabling high-value public and commercial services. These include faster emergency response and situational awareness, routine environmental and agricultural monitoring, and cost-effective inspection of critical infrastructure such as power lines and transport corridors [12]. Strengthening the safety case for UAV integration also supports national bushfire risk reduction and response, where drones are an identified tool for monitoring, assessment, and operations planning [13]. By reducing uncertainty in achievable performance under SWaP constraints, this work contributes evidence that can inform practical requirements and deployment decisions for safe UAV operations in shared airspace [14].

## 1.1 Thesis Structure

**Chapter 2 - Background** introduces the background information related to the following areas: standard radar algorithms, monopulse angle estimation, multi-target tracking, and parallel processing.

**Chapter 3 - Literature Review** conducts an extensive literature review of both foundational and state of the art approaches to performant radar systems under high SWaP constraints. The methodologies of works with similar motivations to ours are critically evaluated for their effectiveness in meeting their aims. Key findings and gaps across works are summarised and an explanation for how this thesis fits in with the existing literature is provided.

**Chapter 4 - Methodology** Explains the methodology by which we compared different radar processing pipeline designs for their suitability for use in detect and avoid systems, and compare different algorithms for their ability to detect small, faraway targets. This methodology will describe the algorithms, the implementations of these algorithms, the processes by which the implementations are profiled, and the processes by which the algorithms are compared for their impact on the overall sensitivity of the system.

**Chapter 5 - Results** Presents the results associated with each of the experiments outlined in the methodology. The results are illustrated as tables and figures, with descriptions of the findings of each of the methodology's stages.

**Chapter 6 - Discussion** The key findings in the results are discussed. The results of different experiment stages are evaluated and explained. Recommendations are provided. The results are tied in with the problem and contributions introduced in this chapter and Chapter 3. The contributions that were not achieved are reflected on.

**Chapter 7 - Project Plan** A summary of what was achieved so far is provided, and a plan for the remaining work is outlined. The plan includes a timeline, stakeholder management plan, and a risk management plan.

CHAPTER 2

# Background

---

UAVs are becoming more abundant and essential across a wide range of applications, including agriculture, inspection, delivery and emergency response. Consequently, standards surrounding Beyond Visual Line of Sight (BVLOS) operation of UAVs are evolving to ensure the safe cohabitation of UAVs with other aerial vehicles within shared airspaces. Aviation standards authorities such as USA's Radio Technical Commission for Aeronautics (RTCA), the European Organisation for Civil Aviation Equipment (EUROCAE) and the International Civil Aviation Organization (ICAO) all incorporated the Airborne Collision Avoidance System (ACAS), an autonomous collision avoidance logic, into their standards [15], [16], [17], reflecting a shift in attention towards the importance of autonomous DAA. In 2025, USA's national aviation authority, the Federal Aviation Administration (FAA) acknowledged this shift by releasing a Notice of Proposed Rulemaking (NPRM) for replacing the waiver-based approval system for BVLOS UAVs with a more capability-based, scalable nationwide framework [18]. Australian regulatory bodies such as Civil Aviation Safety Authority (CASA) are orienting themselves similarly, trialing Temporary Management Instruction (TMI) 2025-03, a streamlined regulatory pathway for certifying BVLOS operations [19].

The ASTM [20], RTCA [21], EUROCAE [22], FAA [18], CASA [19] and ICAO [17] all specify the importance of avoiding both cooperative and non-cooperative aircraft, which are characterised by their use of on-board surveillance transponders like the Automatic Dependent Surveillance-Broadcast (ADS-B). In the presence of non-cooperative targets, a DAA system requires sensors such as radar and optical/infrared cameras to build tracks that ACAS requires. Due to size, weight, power, and compute restrictions, and the variability of hazard size and shapes, building a sensor system for small UAVs that will meet performance targets such as

those outlined in the American Society for Testing and Materials (ASTM) F3442 [20] will be a necessary area of exploration as performance standards and regulatory frameworks are finalised.

## 2.1 Radar Processing

Every radar system operates by transmitting electromagnetic waves into an environment and receiving the reflected signals to detect and locate targets, (Figure 2.1). The strength of the received signal is given by the equation,

$$P_r = \frac{P_t G_t G_r \lambda^2 \sigma}{(4\pi)^3 R^4 L} \tag{2.1}$$

where $P_r$ is the received power, $P_t$ is the transmitted power, $G_t$ and $G_r$ are the gains of the transmitting and receiving antennas, $\lambda$ is the wavelength, $\sigma$ is the radar cross-section of the target, $R$ is the range to the target, and $L$ represents system losses [23].

Hence, the received signal informs us about the target's radar cross-section, reflectivity and range, while the time delay between transmission and reception gives us just range. The Doppler shift of the received signal provides information about the target's radial velocity. If there are multiple receiving channels, the relative phase and amplitude of the received signals across channels can be used to estimate the bearing and elevation of the target.

To measure the range of targets, radar systems need to transmit the signal as pulses. Pulses are generally preferred over continuous constant frequency signal because the time difference between the transmitted pulse and the received echo can be easily measured to estimate range. The transmitted signal is often modulated in frequency or phase. Linearly frequency modulation, (LFM) creates pulses that involve a continuous sweep of frequencies during the pulse duration (see Figure 2.1). Advantages of such a modulation include reduced sensitivity to frequency dependent interference, and improved range resolution due to the effect of pulse compression [24]. Through pulse compression, the long duration LFM pulse can be compressed in time, resulting a signal that illustrates the difference between the transmitted and received signals. One pulse compression technique for LFM signals is described as

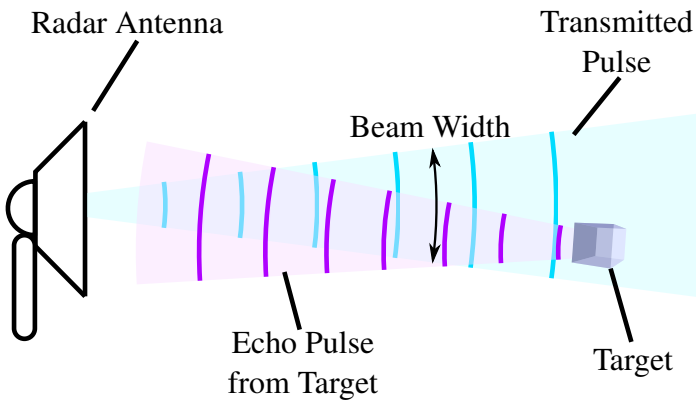



FIGURE 2.1: A illustrative diagram of a radar detecting an object. Objects are detected through the reflection of transmitted radio waves. The time delay and power of the reflected signal provides information about the range and size of the object

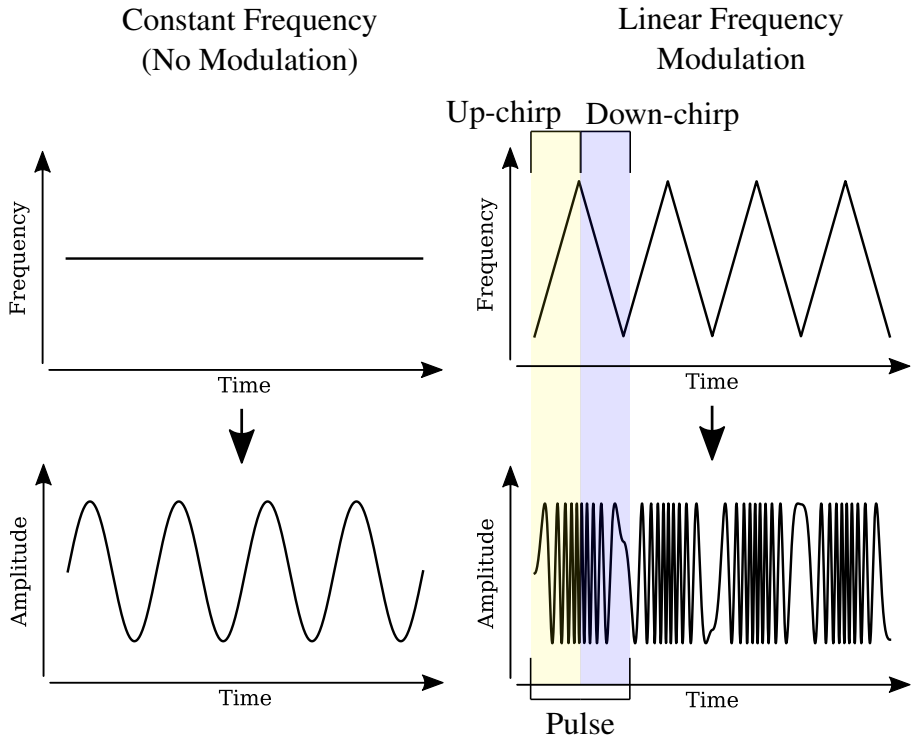
"de-chirping", which involves mixing (term-wise multiplying) the received signal with the transmitted signal to produce a "beat" frequency that gives range information (See Figure 2.1). Modelling each signal as a cosine, the mixed signal's frequencies are given by the product of cosines identity,

$$\cos(\omega_1 t)\cos(\omega_2 t) = \frac{1}{2}[\cos((\omega_1 - \omega_2)t) + \cos((\omega_1 + \omega_2)t)] \tag{2.2}$$

where $\omega_1$ is the transmitted signal's frequency and $\omega_2$ is the received signal's frequency. Since both signals' frequencies increase linearly in time, the difference (beat) frequency $\omega_1 - \omega_2$ is constant and proportional to the time delay and thus the range of the target. By applying a low-pass filter to the mixed signal, we obtain the lower frequency component, which is the beat frequency.

Since the frequency of the transmitted and received signals are linearly increasing in time, knowing the frequency difference between the transmitted and received signals $\Delta\omega = \omega_1 - \omega_2$ allows us to estimate the time delay between the signals and thus the range of the target by the formula,

$$R = \frac{c \cdot \Delta t}{2} = \frac{c \cdot \Delta\omega}{2 \cdot S} \tag{2.3}$$




where $c$ is the speed of light, $\Delta t$ is the time delay between the transmitted and received signals, $\Delta\omega$ is the frequency difference, and $S$ is the slope of the frequency modulation ($\theta/s^2$).

The compressed pulses are often pass through a low pass filter and sampled at a lower rate than the original signal by an Analog to Digital Converter (ADC), attenuating the frequency sum $\cos(\omega_1 + \omega_2)$ while preserving the useful difference frequency $\cos(\omega_1 - \omega_2)$.

Exiting the analogue domain and entering the digital domain, the frequency information in the radar signals can be extracted by applying a Discrete Fourier Transform (DFT) to the sampled data. At this stage the range of clear targets can be estimated by identifying the peaks in the DFT output. At this stage many radar systems also apply a long time Fourier transform on a larger time scale (slow time) to extract the Doppler frequency shift, which provides information on the target's radial frequency.

To improve the detectability of weak targets, radar systems often apply coherent integration across multiple pulses. This involves summing the complex samples across pulses for each

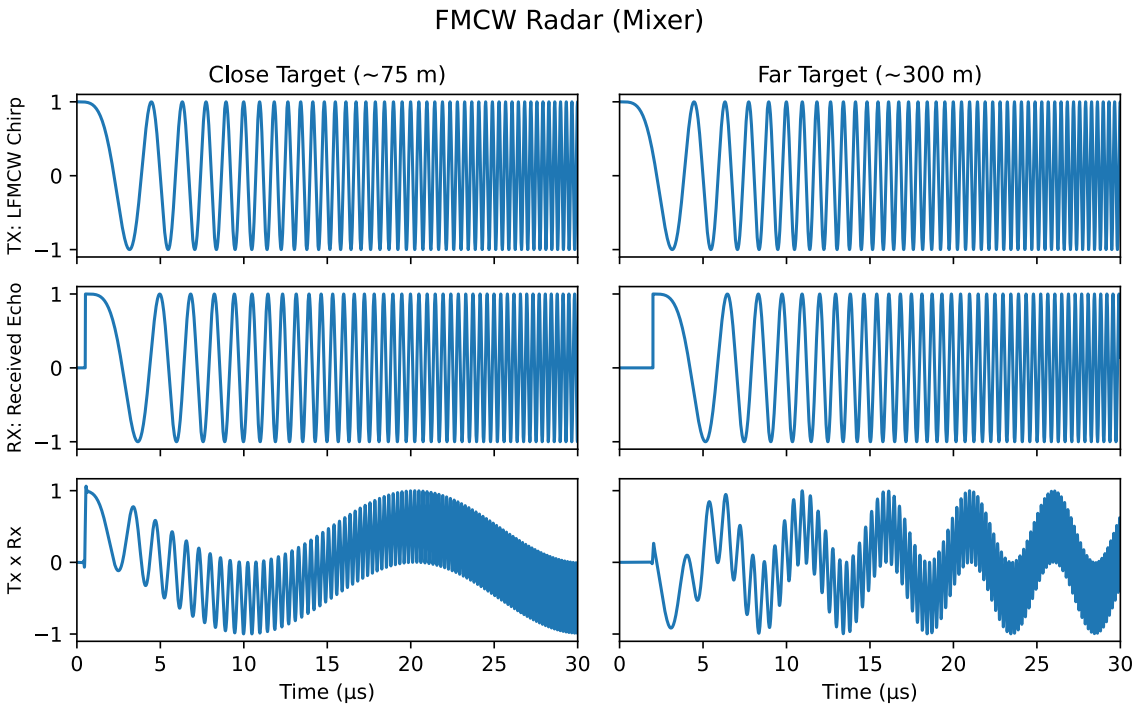



FIGURE 2.2: The transmitted LFM chirp (top), delayed echo (middle), and resulting beat signal after mixing and low-pass filtering (bottom), shown for a close target (left, 0.5 µs delay) and far target (right, 2 µs delay). The far target has a higher beat frequency that the close target.

range bin after the DFT. By summing the complex range bins across multiple pulses, the signal from a target will add constructively while noise will add incoherently improving signal to noise ratio as illustrated in Figure 2.3 [25]. Phase is preserved during integration, which is necessary for later angle estimation stages.

Constant False Alarm Rate (CFAR) detection is a key mechanism for resolving a map of noisy range data into discrete detections given a target false alarm rate. It does this by masking out any samples with magnitude below a threshold determined by the power of the samples around it (see Figure 4.4). This threshold is calculated as,

$$T = \alpha \cdot \frac{1}{N} \sum_{i=1}^{N} x_i \tag{2.4}$$

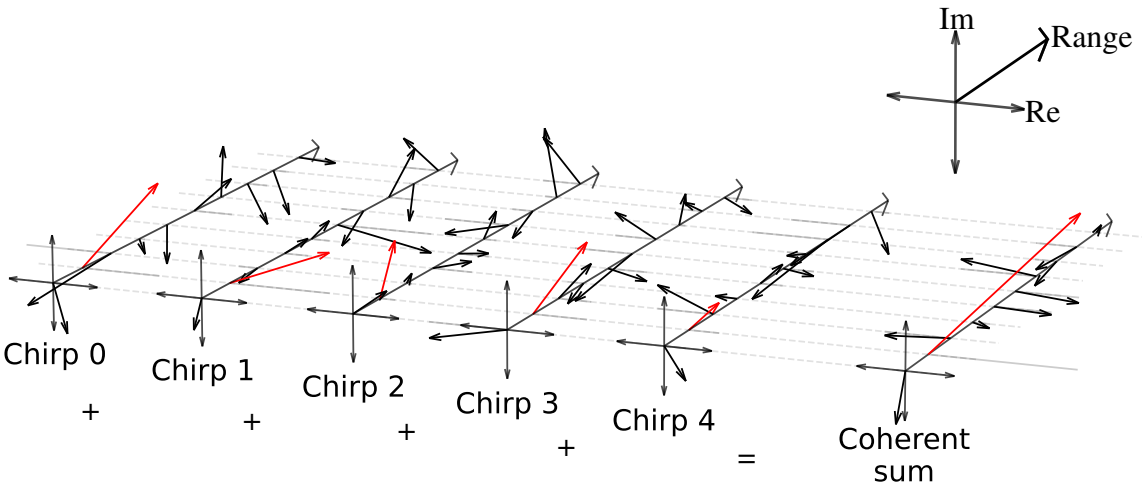



FIGURE 2.3: Coherent integration across five chirps for each range bin. The amplitude of each arrow corresponds to the presence of a target at a corresponding range from the radar, and the argument of the arrow corresponds to the phase of the signal for that range bin. Most bins contain noise-like phasors with varying direction, so their coherent sum remains small. The highlighted bin contains a weak but consistent DC offset across chirps, so its values add constructively, producing a larger resultant value in the coherent-sum.

Where $T$ is the threshold, $\alpha$ is a factor determined by the desired false alarm rate, $N$ is the number of training samples, and $x_i$ are the power (magnitude squared) of the training samples. Under the assumption that the noise in each receiver channel is complex Gaussian, the summed power across four receivers follows a chi-squared distribution with 8 degrees of freedom. For this distribution, the false alarm probability as a function of $\alpha$ is [25],

$$P_{fa}(\alpha; N) = \left(\frac{N}{N+\alpha}\right)^N \sum_{k=0}^{3} \binom{N+k-1}{k} \left(\frac{\alpha}{N+\alpha}\right)^k \tag{2.5}$$

where $k$ is one less than the number of receivers (here $k = 3$, since quadruplet samples from four receivers are summed before thresholding). Figure 2.4 illustrates how CFAR generates detections by comparing a signal to a threshold determined by the power of the surrounding samples.

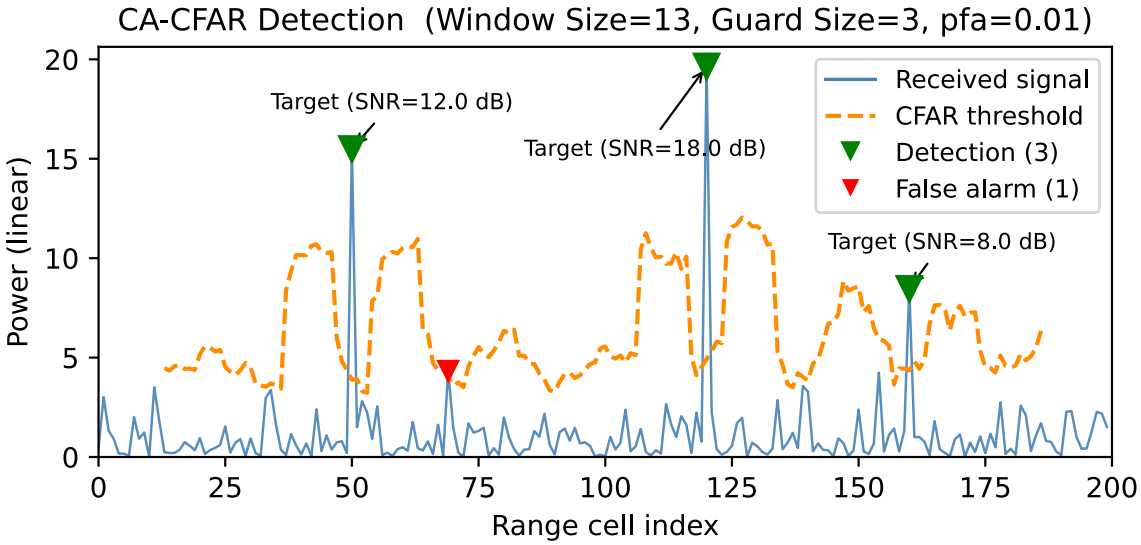



FIGURE 2.4: CFAR detection process. The threshold is calculated based on the power of the samples surround the test cell, and the test cell is compared to this threshold to identify peaks in the signal. The scenario features three target peaks which are each detected by CFAR, while the rest of the signal lies below the CFAR threshold, excluding one outlier noise sample.

## 2.2 Monopulse Angle Estimation

Target angle sensing is one of the principal functions of radar systems. In early systems, scanning or sequential lobing was used to estimate angle, which involved mechanically steering the radar beam and comparing the signal strength across different beam positions. Monopulse systems on the other hand use multiple beams simultaneously to estimate angle using the difference in amplitude or phase across each of the received channels [26].

In this thesis, we focus on amplitude comparison monopulse processing, which is commonly used in small radar systems due to its size and weight requirements. About a particular dimension, the angle of a target can be estimated with the equation,

$$\theta = f\left(\frac{S_1 - S_2}{S_1 + S_2}\right) \tag{2.6}$$

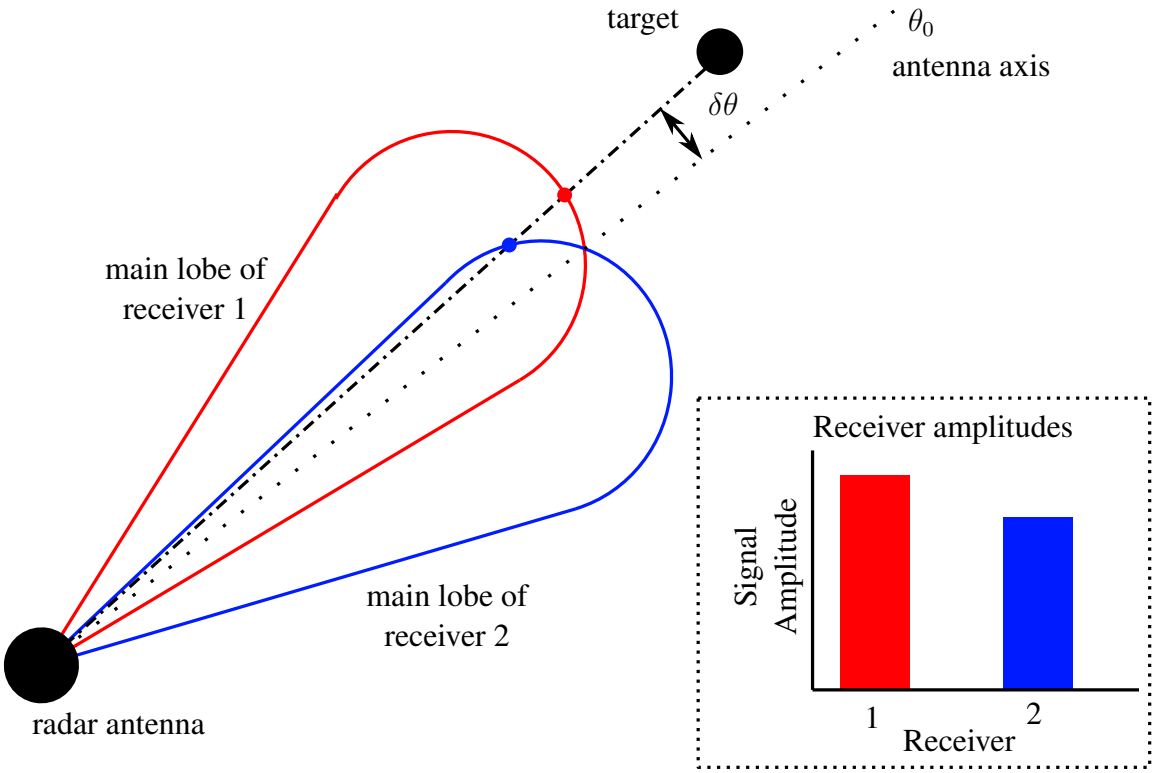



FIGURE 2.5: Principle of amplitude comparison monopulse processing. The angle of the target is estimated as near the antennae axis slightly to receiver 1 because the amplitude from receiver 1 is stronger than from receiver 2.

where $S_1$ and $S_2$ are the signal strengths in the two channels being compared, and $f$ is a function that maps the sum-difference ratio to an angle. This explanation is illustrated in Figure 2.5.

With four receiver channels, bearing and amplitude estimates can be found in two dimensions by comparing the sum-difference ratios across two pairs of channels. The bearing and elevation estimates are given by,

$$\delta_b = \frac{|a_{0,0} + a_{1,0}| - |a_{0,1} + a_{1,1}|}{|a_{0,0} + a_{1,0}| + |a_{0,1} + a_{1,1}|} \tag{2.7}$$

$$\delta_e = \frac{|a_{0,0} + a_{0,1}| - |a_{1,0} + a_{1,1}|}{|a_{0,0} + a_{0,1}| + |a_{1,0} + a_{1,1}|} \tag{2.8}$$

The relationship between the ratios and true angles is dependent on the specific radar system. Different receivers will have different gains. Calibration allows us to map the sum-difference ratios to angle estimates in radians using two polynomial functions of the form,

$$\theta_b = \sum_{i=0}^{4} \sum_{j=0}^{4-i} c_{bij} \delta_b^i \delta_e^j \qquad \theta_e = \sum_{i=0}^{4} \sum_{j=0}^{4-i} c_{eij} \delta_b^i \delta_e^j \tag{2.9}$$

One for bearing and one for elevation, where $c_{bij}$ and $c_{eij}$ are the coefficients obtained from calibration data.

## 2.3 Multi Target Tracking

Multi-target tracking (MTT) converts a sequence of noisy radar detections into stable estimates of target state over time. Each radar update produces a set of detections characterised by range, bearing, and elevation, as described in the preceding sections. The role of the tracker is to associate these detections with targets across successive updates, filter out noise, and maintain a continuous estimate of each target's state to support detect-and-avoid (DAA) decision making.

A tracking filter combines a model of target motion called the transition model with incoming measurements to produce an evolving state estimate over time. The Kalman filter [27] is a standard framework, alternating between a prediction stage, in which the current state is propagated forward using a motion model, and an update stage (see Figure 2.3), in which a new measurement corrects the prediction. A constant velocity (CV) with noise is a common motion model, and is represented in three dimensions with a matrix for the state transition and a covariance matrix for the process noise. For manoeuvring targets the interacting multiple model (IMM) [28] maintains a bank of filters under different motion assumptions and blends their outputs probabilistically.

Since radar measurements are expressed in spherical coordinates while motion models are most naturally formulated in Cartesian space, the measurement equation is nonlinear. The

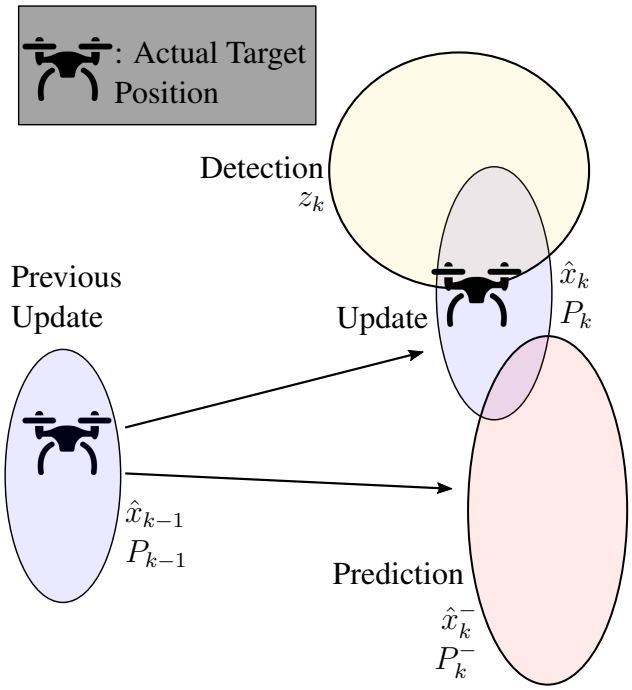



FIGURE 2.6: Each time step, the makes a prediction of the target's current state based on the previous state and a motion model (red), and then updates the prediction based on the new measurement (yellow) to produce an improved estimate (blue). The detection, prediction and updated estimation are each represented as a Gaussian distribution, hence the ellipses, with the mean representing the most likely state and the covariance representing the uncertainty.

Extended Kalman Filter (EKF) handles this via first-order linearisation, while the Unscented Kalman Filter (UKF) [29] propagates sigma points through the nonlinear transformation for improved accuracy when the nonlinearity is significant.

In the context of multi-target tracking, the Kalman filter can not be used until detections have been associated with targets. To determine which detections correspond to which targets a common strategy involves gating target candidates for each detection by finding the distance between the predicted measurement for each target, and the actual measurement. After gating, and association algorithm such as the Global Nearest Neighbour (GNN) algorithm can create a one-to-one assignment between detections and targets.

In dense clutter, hard assignment methods can be insufficient. Probabilistic data association (PDA) updates each track as a weighted combination of all gated detections, with weights

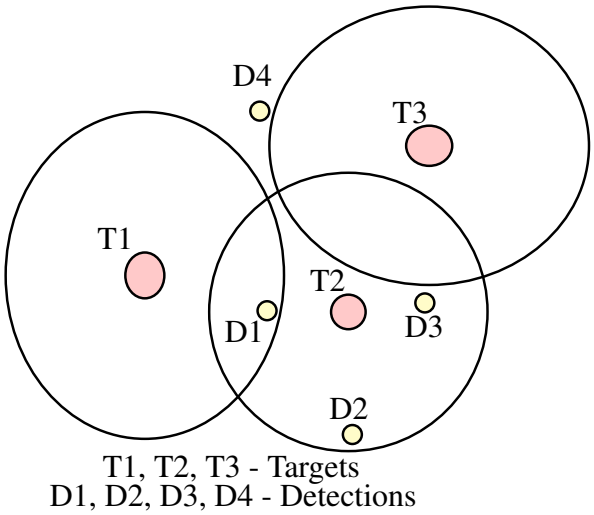



FIGURE 2.7: Global nearest neighbour data association algorithm associating three targets measurement predictions in red with four detections in yellow. The distance based gates are depicted as ellipses around the predicted measurements. The algorithm finds a one-to-one assignment between targets and detections by minimising the overall distance between predicted measurements and actual measurements. In this case, the assignment would be T1-D1, T2-D3 and T3 with no detection.

proportional to measurement likelihood. Joint PDA (JPDA) extends this to the multi-target case [30]. Multiple hypothesis tracking (MHT) defers the association decision by maintaining a tree of hypotheses across multiple scans [31] (see Figure 2.3).

Detections that cannot be associated with any existing track initialise tentative tracks, which are promoted to confirmed only after receiving at least M detection associations in the last N timesteps, where M and N are parameters of the promotion policy, commonly referred to as M-of-N logic [32]. This prevents transient clutter returns from generating false confirmed tracks. Conversely, tracks accumulating consecutive missed detections are allowed to coast for a limited number of updates before being deleted. The promotion and deletion thresholds balance responsiveness to new targets against robustness to clutter and intermittent detection loss and is generally tuned to fit the operating environment.

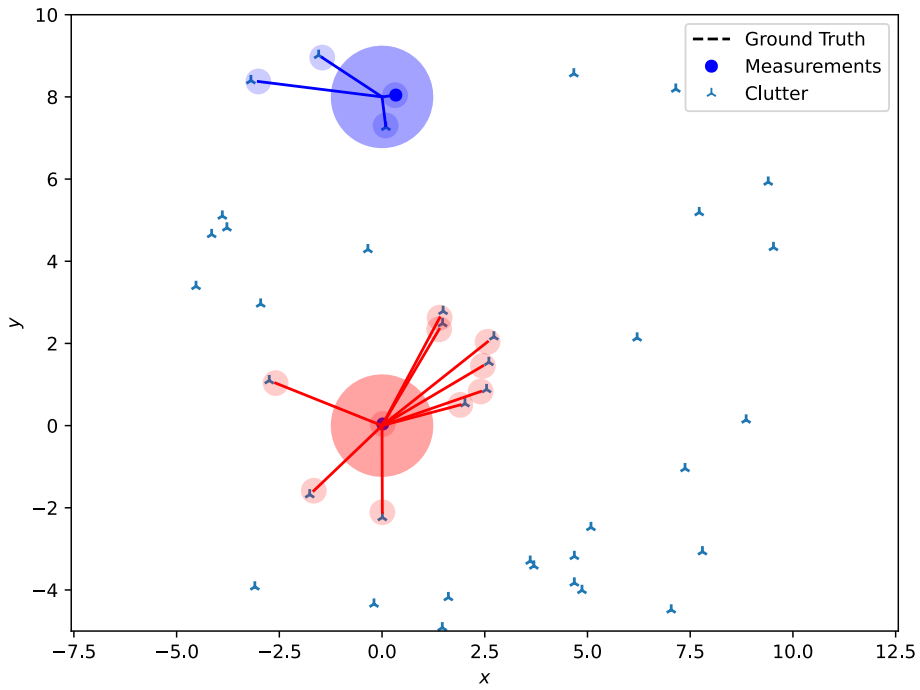



FIGURE 2.8: Multiple hypothesis tracking algorithm maintaining branching tracks of two targets (in blue and red) with their measurements indicated by blue dots. "Clutter" detections that make the path of the targets unclear are indicated as triangles. The blue and red shaded circles represent several potential posterior estimates of each of the two targets at the current timestep, reflecting the ambiguity in associating the measurements to the targets amongst the clutter. There is also an uncertain "no detection' hypothesis for each target indicated by the large circles. The size of the shaded circles indicate 3 standard deviations of uncertainty.

## 2.4 Parallel Processing

Many stages of a radar signal processing pipeline — range compression, coherent integration, CFAR, and angle estimation — involve applying the same operation independently across large volumes of data. CPUs, GPUs, FPGAs, and ASICs each offer different trade-offs between flexibility, throughput, power efficiency, and development effort.

A GPU executes a large number of lightweight threads (Figure 2.9), organised into warps, groups of 32 threads that execute the same instruction simultaneously on a single streaming multiprocessor (SM). This single-instruction multiple-thread (SIMT) model maps directly onto operations such as FFTs, element-wise multiplication, and threshold comparisons, where

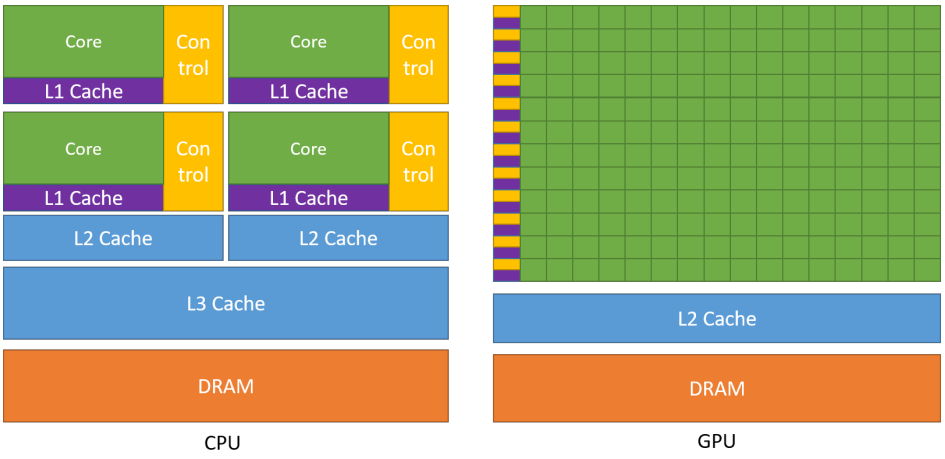



FIGURE 2.9: The GPU devotes more transistors to data processing than the CPU. Each (green) core acts as a thread that executes the same instruction on different data elements. Source: [33].

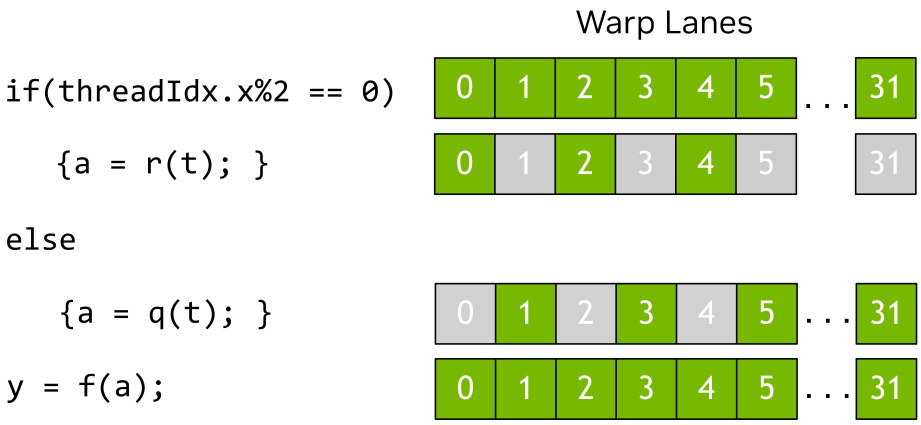



FIGURE 2.10: In this example, branching paths in execution between the even and odd threads prevent all threads from executing in parallel. All even threads execute the top branch while all odd threads are idle, then vice versa. As a result, the execution time is the sum of the two branches instead of the maximum. Source: [33].

the same computation is applied independently to many data elements. When threads within a warp follow different control flow paths — a condition known as warp divergence — the divergent branches must be executed serially, reducing parallelism (see Figure 2.10).

Memory access patterns also affect throughput. Global memory is accessed most efficiently when threads in a warp read or write contiguous memory locations simultaneously, a pattern known as coalesced access. Non-coalesced access doesn't utilise memory bandwidth

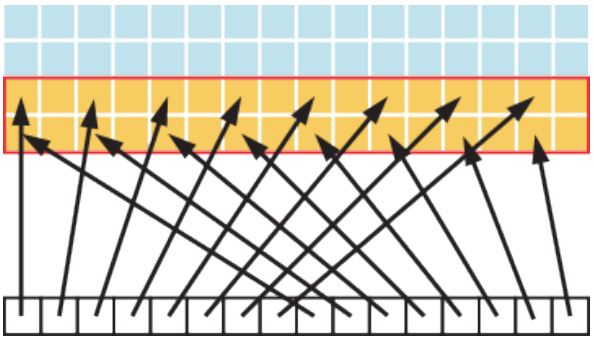


FIGURE 2.11: Figure 6: Adjacent threads (bottom) accessing every second memory location (orange). Since memory is cached in large blocks, the large stride of 2 results in a 50% reduction in load/store efficiency, since half of all cached memory is not being used. Source: [33].

efficiently by not accessing contiguous memory locations. Figure 2.11 illustrates one such situation. The warps will need to retrieve data from global memory more frequently when accessing non-contiguous memory. For this reason, ensuring that as much as possible of the data in each cache line fetched is actually used is an important part of performance optimisation of memory accesses on these devices.

In discrete GPU systems, the CPU and GPU maintain separate physical memory pools connected by a bus such as PCIe, and data must be explicitly transferred between host and device memory. In system-on-chip (SoC) platforms such as the NVIDIA Jetson family, the CPU and GPU share the same physical DRAM. This is called physically shared memory. Both processors access the same physical addresses, and the memory subsystem handles coherency, eliminating the explicit transfer step.

Shared memory is a separate, small, low-latency scratchpad memory local to each SM, explicitly managed by the programmer. It allows threads within a thread block to exchange data and cache intermediate results without accessing global memory. It is used in GPU implementations of reductions, transpose operations, and tiled matrix multiplications, where threads cooperatively load a tile of data into shared memory and reuse it across multiple operations.

CHAPTER 3

# Literature Review

## 3.1 Compact Airborne Radar for UAV DAA

Early demonstrations established that capable radar sensing is achievable under UAV SWaP constraints. Sysak et al. [34] present a lightweight X-band monopulse radar for mid-air collision avoidance on medium UAV platforms, integrating a multi-channel forward-looking antenna with FPGA-based pulse compression on a single 60 gram PCB, with average power below 80 Watts. Field trials demonstrate reliable detection of large obstacles at 1–3 km, and the work validates a simple online pipeline from range compression to TCPA estimation. However, validation is conducted exclusively against large, high radar cross-section targets, and system latency is not reported. This thesis addresses these gaps by characterising detection sensitivity against small moving airborne targets and by measuring end-to-end pipeline latency on embedded hardware.

Wang et al. [7] demonstrate a complete low-SWaP LFMCW system for group 2/3 UAS incorporating FFT-based detection, digital beamforming, multi-target tracking, and GPU-accelerated FFT for update rate improvement. However, performance is validated in simulation only, with no latency measurements, no throughput figures, and no benchmarking of the GPU claim. This thesis addresses these gaps by providing hardware-validated latency and throughput measurements for a comparable pipeline, and by substantiating the GPU acceleration argument with systematic benchmarks.

Moses et al. [35] introduce a 230 gram miniature radar mountable on a small quadcopter, demonstrating that radar hardware can be integrated onto very small platforms. Their evaluation emphasises target identification via micro-Doppler signatures rather than ranging or tracking, leaving open questions about how the small aperture affects range-domain SNR, range resolution, and false alarm behaviour. This thesis addresses this gap by benchmarking range–Doppler processing, CFAR detection, and multi-target tracking under the SNR conditions characteristic of small-aperture UAV radars.

## 3.2 Embedded Radar Processing Under SWaP Constraints

Shi et al. [36] describe a small FMCW phased array built from off-the-shelf development boards, validating that continuous real-time 2D FFTs are achievable on a 100 MHz FPGA. However, processing was split between an onboard FPGA and offboard post-processing, and signal coherence problems prevented real-world validation. This thesis addresses this gap by implementing a fully onboard, online pipeline and validating it against real target returns.

Newmeyer et al. [37] demonstrated a 120 gram, 8 Watt FMCW phased array pipeline on a Zynq ARM CPU, closely monitoring power, latency distributions, and hardware resources. This work clearly establishes that radar DAA can operate under strict SWaP constraints. However, only one hardware configuration and one algorithm sequence are evaluated. This thesis addresses this gap by systematically comparing alternative algorithmic pathways and evaluating how pipeline bottlenecks change across embedded compute configurations, including GPU-based architectures.

Zhao et al. [38] implement CUDA acceleration for a passive bistatic radar pipeline, reporting up to a 113.13$\times$ speedup over a CPU baseline across three algorithm stages. However, the study covers only clutter suppression, range–Doppler processing, and CA-CFAR detection, omitting angle estimation, coordinate transforms, and Kalman filtering, and does not discuss memory reallocation or thread synchronisation costs. This thesis addresses these gaps by benchmarking a more comprehensive algorithm selection and profiling host–device transfer and synchronisation overhead explicitly.

Zaknoun [39] provides a comprehensive treatment of GPU radar software architecture for real-time processing. However, this work is not situated in a UAV DAA context and does not evaluate performance under UAV SWaP constraints. This thesis adapts these architectural insights to the UAV DAA problem, evaluating how conclusions change under constrained power and memory budgets.

## 3.3 Sensitivity and Detection in Low-SNR Regimes

A key consequence of SWaP-constrained hardware is reduced aperture, which simultaneously lowers SNR and degrades angular resolution, increasing the likelihood that multiple targets occupy the same resolution cell.

An et al. [40] propose an efficient method for resolving two unresolved targets within a single monopulse cell. However, evaluation is limited to simulated white-noise conditions, the method is not integrated into an end-to-end range–Doppler pipeline, and explicit failure modes exist for same-angle targets and cells containing three or more targets. This thesis addresses these gaps by testing the algorithm under non-white noise conditions and within a complete tracking pipeline, measuring the effect on track quality.

Aubry et al. [41] demonstrate that detection and angle estimation can be treated jointly in a multichannel array, increasing sensitivity to targets not visible in any single channel and avoiding the brittleness of early thresholding under low SNR. However, validation is limited to single simulated targets in Gaussian interference. This thesis addresses this gap by applying the algorithm to multiple targets under non-Gaussian noise regimes and measuring both sensitivity and track-quality impacts alongside compute cost.

The noise and clutter regime experienced by a miniature airborne radar has not been empirically characterised in the literature. Ezuma et al. [42] demonstrate that Gaussian and Rayleigh clutter assumptions — the basis of CA-CFAR — already fail for ground-based mmWave radar detecting UAVs, finding that a Weibull distribution provides a significantly better fit to the measured clutter statistics, and explicitly noting that prior UAV detection work had

neglected this. The airborne case, where the radar sensor is itself mounted on a vibrating UAV platform subject to altitude variation, aspect angle changes, and platform-induced interference, introduces further departures from the homogeneous Gaussian noise model that underpins standard windowing and CFAR algorithm selection. No published work has measured or modelled these statistics for a miniature airborne FMCW radar in real flight conditions. As a result, the algorithm choices made in existing pipeline designs, (window function selection, CFAR variant, and training cell configuration) lack empirical justification for this operating regime.

## 3.4 Summary and Research Gap

Across the UAV DAA radar literature three gaps emerge. First, existing hardware demonstrations validate detection against large, high radar cross-section targets only; no work characterises sensitivity to small moving airborne targets under realistic encounter conditions. Second, processing feasibility on constrained embedded hardware has been demonstrated on FPGAs and ARM CPUs, but no work benchmarks a GPU-based pipeline in a UAV DAA context with comprehensive algorithm coverage and architectural profiling. Third, the noise and clutter regime of miniature airborne FMCW radar has not been empirically characterised. Existing work shows that Gaussian clutter assumptions fail even for ground-based mmWave radar, yet no study has measured these statistics for an airborne sensor. As a result, the window function, CFAR variant, and training cell choices made in existing pipeline designs lack empirical justification for this operating regime, and angle sensing algorithms have only been validated under idealised white Gaussian noise with single simulated targets.

This thesis addresses all three gaps by characterising detection and tracking sensitivity against small moving airborne targets, by benchmarking a comprehensive selection of radar processing and tracking algorithms on embedded GPU hardware with explicit profiling of host–device transfer and synchronisation overhead, and by collecting real flight data to characterise the noise regime and evaluate monopulse angle sensing algorithms within a complete multi-target tracking pipeline under realistic conditions.

CHAPTER 4

# Methodology

## 4.1 Overview

This thesis designs, implements, and evaluates a software-defined radar processing pipeline for a small monopulse radar system, assessing both computational efficiency and tracking accuracy on an NVIDIA Jetson Xavier NX embedded GPU platform. The methodology is structured to directly address the gaps identified in the literature review: the absence of comprehensive systems-level evidence of algorithm selection, low level implementation trade-offs, and hardware-aware design conclusions under SWaP constraints. Through the methodology outlined in this chapter, we fulfill the contributions of this thesis, to compare radar algorithm configurations by sensitivity and track quality, to compare low-level implementations by computational performance, and to derive hardware-aware design recommendations.

The software processing pipeline is designed to receive the radar data from four Texas Instruments AWR2243 radar chips, and produce tracks of targets in the radar's field of view. It is designed to be modular, with each stage of the pipeline performing a specific function. The pipeline is comprised of three subsystems, a pre-processing subsystem which is concerned with preparing the raw radar data for detection, an aggregator subsystem, which is concerned with combining the processed data from the four video streams into a single stream, and a detection and tracking subsystem, which is concerned with detecting targets in the radar data and tracking them over time. This subsystem level design is depicted in Figure 4.1.

The methodology is organised into three parts. Firstly, we propose a design of a (pre-)processing subsystem that performs formatting, windowing, FFTs, and coherent integration on dense

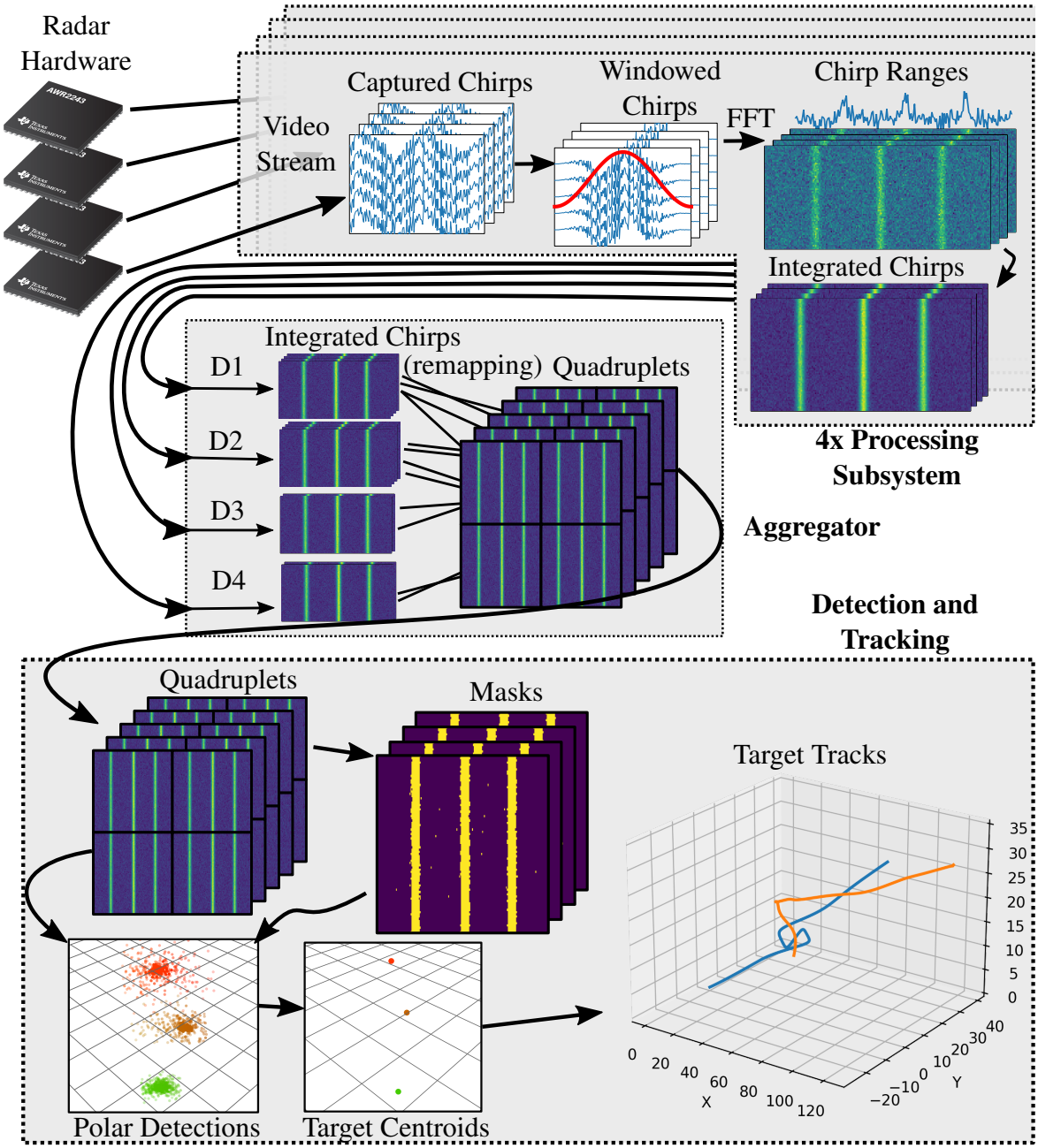



FIGURE 4.1: System level dataflow diagram. The pre-processing subsystem receives the AWR2243's raw radar data over four CSI video streams. The data is organised into frames, each containing many chirps. The processing subsystem then outputs the data in the form of chirps containing target range information. The aggregator subsystem receives the four resulting processed data streams, and combines them into a single data stream organised by quadruplet, a collection of four receivers necessary for monopulse angle sensing. The detection and tracking subsystem receives a stream of quadruplets, and produces tracks of targets in the radar's field of view.

radar data. Then, we propose the design of a novel detection and tracking subsystem that performs CFAR, monopulse angle sensing, clustering, and multi-target tracking. Lastly, we describe the datasets used for testing.

## 4.2 Processing Subsystem Evaluation

The proposed preprocessing subsystem is responsible for receiving the raw radar data from the AWR2242 radar chips, and preparing the data for detection. An instance of the subsystem runs per incoming video stream. The subsystem performs the following operations sequentially: formatting, windowing, FFT, and coherent integration, (see Figure 4.2). Below are the implementation and experiment details for each of the operations.

### 4.2.1 Windowing

To reveal target range from a single radar pulse, an FFT (Fast Fourier Transform) is applied to the de-chirped pulse. The DFT assumes the finite signal repeats periodically, so when the signal does not start and end at the same value, the implicit periodicity introduces a discontinuity. This causes energy from a true frequency to spread into adjacent bins — a phenomenon known as spectral leakage.

When an FFT is performed on a finite signal, it is implicitly multiplied by a rectangular window. A term-wise multiplication in time corresponds to a convolution in the frequency domain, and the rectangular window's sinc-shaped frequency response has large sidelobes that spread energy from each frequency bin into its neighbours. By applying a more suitable window function prior to the FFT, spectral leakage can be reduced, as shown in Figure 4.3.

This thesis evaluates a range of practical window functions and their implementation on CPU and GPU. Candidate windows include common real-time tapers such as Hann and Hamming, lower-sidelobe windows such as Blackman and Blackman–Harris, and adjustable-parameter windows such as Kaiser, which allow explicit control over sidelobe level and mainlobe width. Because windowing is a point-wise multiplication, it is naturally suited to single instruction,

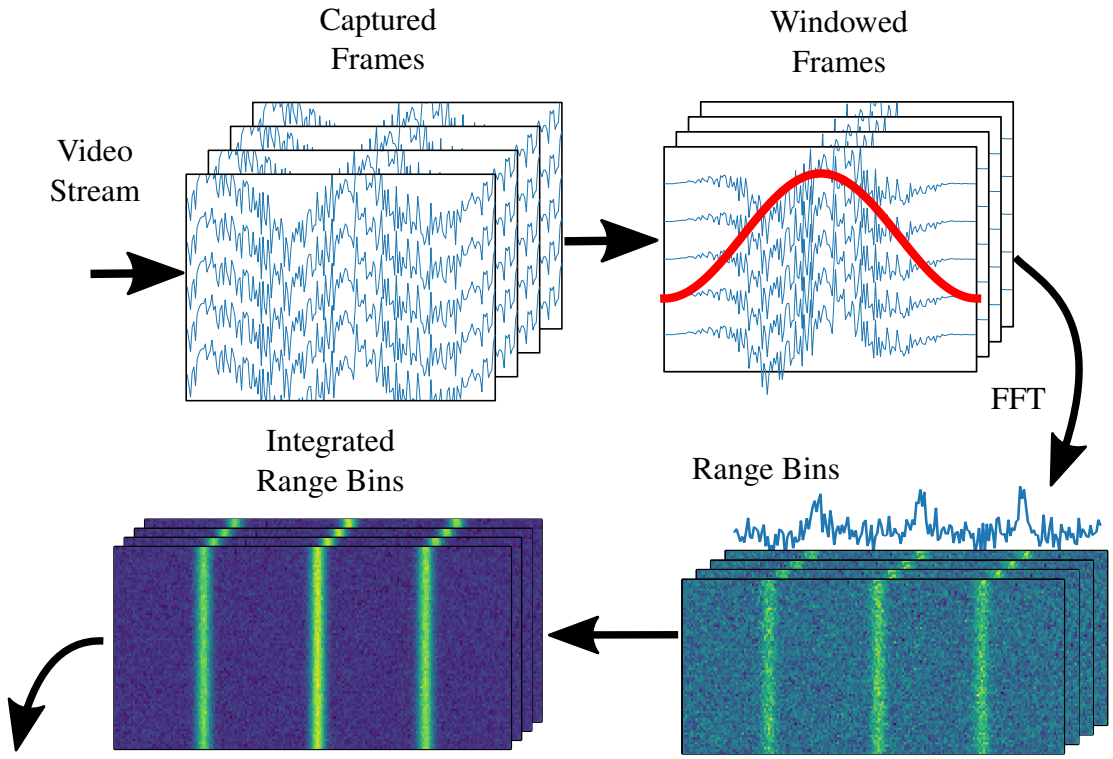



FIGURE 4.2: Dataflow for the preprocessing subsystem. Multiple instances of the subsystem run on separate processes, with each instance receiving compressed baseband radar data samples from an incoming video stream. Formatting organises samples into contiguous blocks in memory, and discards empty chirps. The windowing stage applies a window function to the data to reduce spectral smearing in the next stage. The FFT stage applies a Fourier Transform across fast time into the frequency domain, revealing target range. The coherent integration stage sums the complex FFT values of identical pulses within the same frame.

multiple data (SIMD) execution. These implementations are compared for their resulting spectral behaviour.

Computational performance profiling measures the latency contribution and throughput associated with each window and implementation method. A cost-benefit analysis weighs the latency cost against downstream benefits of a cleaner signal, including reduced sidelobe-induced false positives. These procedures provide a basis for selecting windows that are spectrally well behaved, computationally efficient, sensitive, and positively affect the ending track quality.

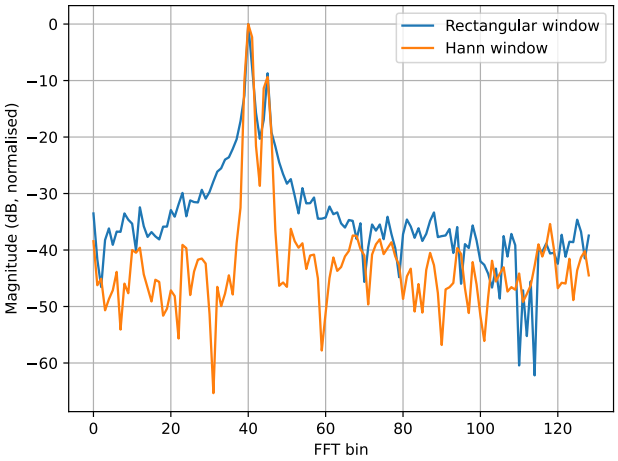



FIGURE 4.3: Results of an FFT performed on a signal with two frequencies and Gaussian noise, with and without a Hann window. Without the Hann window, the bins close to the peak frequency bins 40 and 45 are significantly elevated due to spectral leakage from those peaks, which would cause nearby bins to be falsely detected as targets.

### 4.2.2 Coherent Integration

Different integration window sizes are compared on real data, and the resulting changes in detection sensitivity and false alarm behaviour are measured. CPU and GPU implementations are also compared, and the downstream effect on angle sensing stability is quantified. An ablation study holds the window, FFT size, and CFAR parameters fixed while varying only the integration policy, isolating its effect. The contribution of the integration stage to the overall latency and throughput of the pipeline is also measured.

## 4.3 Tracking Application Evaluation

### 4.3.1 CFAR

For finding the CFAR coefficient $\alpha$ (defined in 2.4) that gives a desired false alarm rate $P_{fa}^{\star}$, we cannot rearrange the equation into a closed-form expression.

Hence, for a desired false alarm rate $P_{fa}^{\star}$, $\alpha$ is found by solving

$$P_{fa}(\alpha; N) = P_{fa}^{\star} \tag{4.1}$$

using bisection. The method begins with $\alpha_{lo} = 0$ and increases an upper bound $\alpha_{hi}$ until $P_{fa}(\alpha_{hi}; N) \leq P_{fa}^{\star}$. It then repeatedly bisects the interval and evaluates $P_{fa}(\alpha; N)$ at the midpoint, updating the lower or upper bound depending on whether the resulting false alarm rate is above or below the target. This continues until the interval is sufficiently small. In this way, $\alpha$ is obtained as the numerical solution that gives the required false alarm rate.

Additionally, three different implementations of CFAR are compared for their latency, a naive implementation on the CPU, an implementation that uses a prefix sum table, and a naive solution on the GPU. Additionally different noise assumptions are tested. The naive implementation involves calculating the noise statistic with a repeated sum per sample of interest. The prefix sum table method reduces the number of sums required to four per sample of interest regardless of the number of training cells (see Figure 4.3.1). The GPU implementation creates a thread per sample of interest (see Figure 4.3.1). A sweep of window size, frame buffer size and false alarm rate values is conducted for each CFAR implementation on real and simulated data. The performance (throughput, latency, memory, false alarm rate, sensitivity) is compared across configurations in a cost benefit analysis. This addresses the literature gap in understanding how detection sensitivity and false alarm behaviour trade off against latency and throughput under SWaP constraints.

### 4.3.2 Monopulse Angle Sensing

Angle sensing converts multi-channel measurements into angle estimates that drive association and tracking. We assess candidate angle discriminators by running controlled sweeps over SNR, channel imbalance, and multi-target proximity, and measuring angular bias, mean squared error, and confidence. The latency and throughput of CPU and GPU implementations are measured. The trade off being software implementation details such as warp divergence, and instruction-level parallelism.

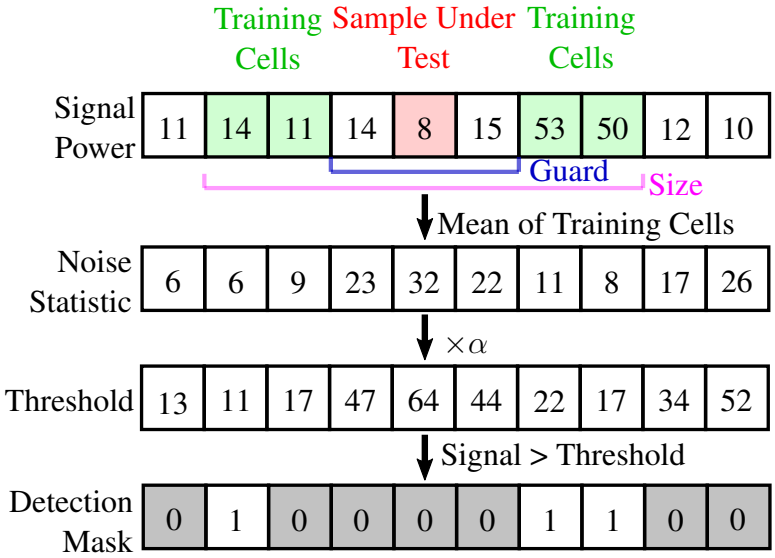



FIGURE 4.4: Naive Implementation of CFAR for the CPU. Each sample is compared to a threshold determined by the power of the samples around it. The threshold is calculated by taking the average power of the surrounding samples and multiplying it by a factor $\alpha$ determined by the desired false alarm rate.

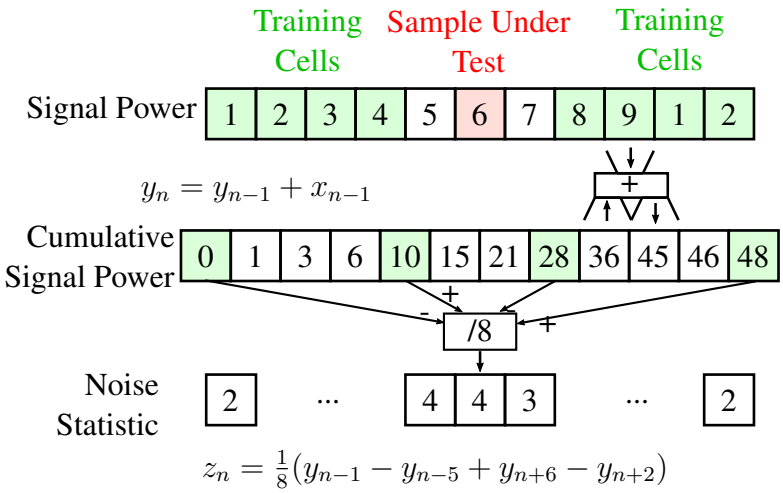



FIGURE 4.5: CFAR implementation using a prefix sum table (cumulative signal power). The prefix sum table allows the noise statistic to be calculated with a fixed number of sums per sample of interest, regardless of the number of training cells. In this specific case, after the cumulative signal power is calculated in preprocessing, it only takes four sums of to find the sum of eight training cells.

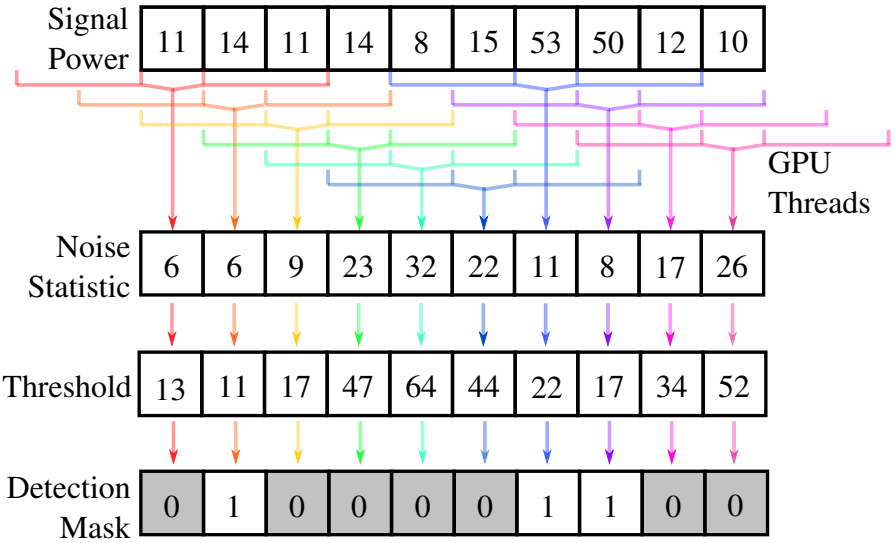



FIGURE 4.6: CFAR implementation on the GPU. Threads (each depicted as a unique colours) are each responsible for a sample, allowing for parallel processing of the CFAR algorithm across the entire range profile. Each thread calculates its own noise statistic, threshold, and mask.

The angle sensing algorithm features a few early returns that result in warp divergence on the GPU. For example, if the bearing and elevation ratios are outside of the domain over which the polynomial fit is valid. Instruction level parallelism is a measure of how many instructions can be executed in parallel on a single thread. By changing the order of operations, we can potentially increase instruction level parallelism, potentially reduce the latency of the GPU implementation.

## 4.4 Testing Datasets

The datasets described below are used for the experiments outlined in the previous sections (4.2 and 4.3).

The first dataset is used to validate both the correctness and the speed of our processing algorithms during development. By generating binary data streams containing complex sine waves with Gaussian noise using bash scripts, we can test the radar software under controlled conditions where the expected output can be precomputed in advance. This makes it possible

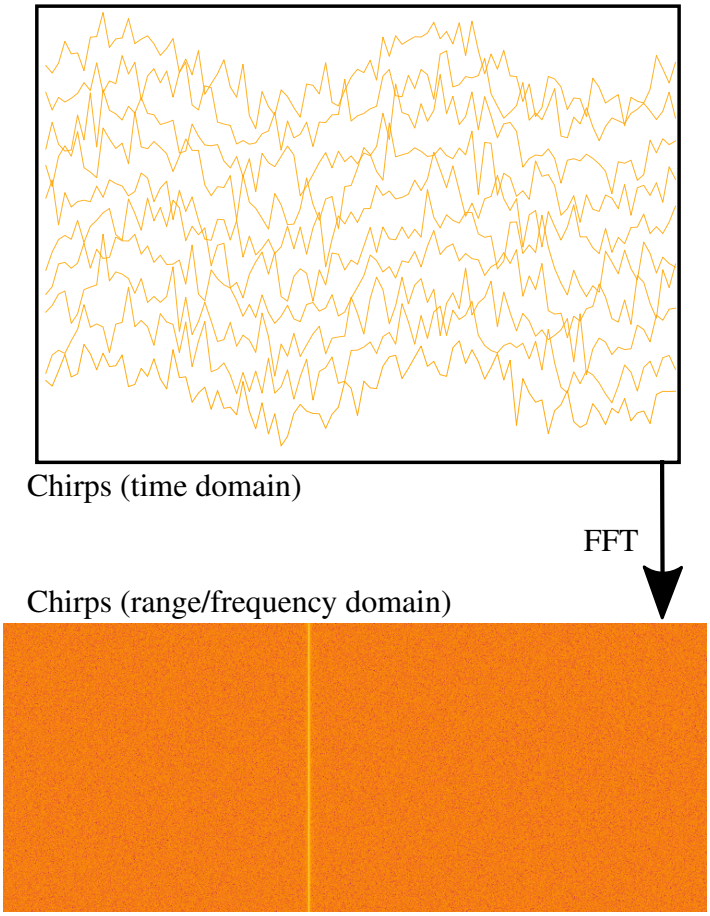



FIGURE 4.7: An example of a synthetic signal generated for testing. The top plot shows the real components of the samples of many received chirps (stacked vertically). Fast time increases to the right and the height of a sample within a chirp is its voltage. In the bottom plot, the horizontal axis is now range, successive chirps are stacked vertically and amplitude is depicted with colour, where yellow is the highest amplitude. 2nd plot is produced by applying an FFT to each synthetic chirp from the first plot. The transform allow us to visually validate that the frequency to range conversion is correct, and the preprocessing stage is behaving as expected.

to verify algorithmic correctness while also evaluating latency and throughput without the additional complexity introduced by real-world phenomena such as multipath propagation, data corruption, and non-gaussian noise. An interactive variation of this dataset is also used for debugging, allowing the user to adjust parameters such as SNR, range, and bearing/elevation ratios in real. Figure 4.7 illustrates an example of a synthetic signal generated for testing.

The second dataset consists of recorded data from the AWR2243 radar system operating in `TEST_SOURCE` mode. This dataset is used to validate the behaviour of the processing pipeline under more realistic data rates, timing uncertainties, and electrical noise conditions. In `TEST_SOURCE` mode, the chip can simulate two targets with configurable position, velocity, and bounds, and the receivers capture the signals that would be reflected from these targets. Although this recorded data is still not fully representative of real-world radar returns, it provides a more realistic approximation of practical operating conditions than the synthetic sine wave dataset.

CHAPTER 5

# Results

## 5.1 Overview

This chapter presents the validation process and results of the processes outlined in the Methodology (Chapter 4). The results are organised by operation. Each section discusses the correctness and computational performance of an algorithm, as well as the results of the ablation studies. The results are presented as plots, and are accompanied by a discussion of their implications for the design of the radar processing pipeline in the following chapter.

## 5.2 Windowing

The Hann and Hamming windows are first evaluated for the prominence of their main lobe and sidelobes in the frequency domain on the synthetic datasets. Figure 5.1 illustrates the performance of each of the filters on a clear sine wave with no noise. Figure 5.2 illustrates the performance of each of the filters on the noisy simulated data with an SNR of 1.

In terms of computational performance, the latency contribution of the filters does not significantly differ across implementations. Figure 5.3 shows the time taken to process a single frame of 7 670 000 samples, averaged over 50 runs.

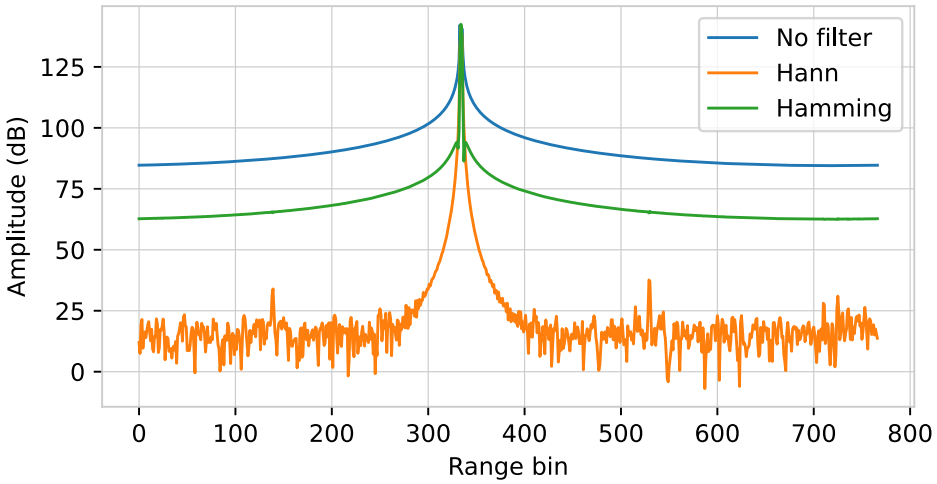



FIGURE 5.1: Results of an FFT performed on a single frequency signal without noise without a filter, with a Hann window, and a Hamming window. Each of the tests correctly produces a peak near bin 334, the target range. The no filter option's peak is wider than the filtered options. The Hann window performs the best globally, with a 100 dB reduction in power when more than 40 bin away from the target bin. The Hamming window performs significantly worse globally, with only a 70 dB reduction when more than 40 bins away, but has a steeper drop-off near the target bin, which may result in better resolution of closely spaced targets.

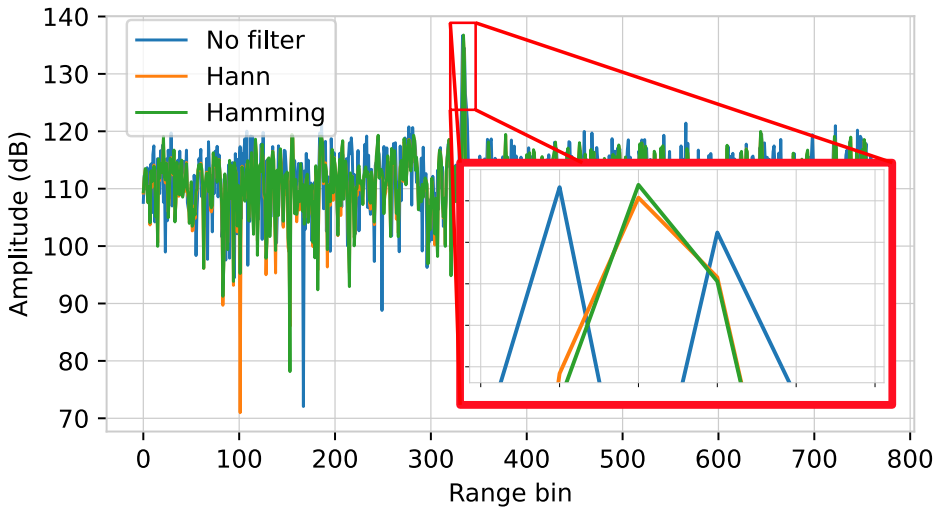



FIGURE 5.2: Results of an FFT performed on a single frequency signal with an SNR of 1 with no window filter, with a Hann window filter, and a Hamming window filter. The no filter option's spectral leakage acts to deform the peak at bin 334. The Hann and Hamming windows perform similarly globally but the Hamming window features a slightly higher peak, and less spectral leakage into the adjacent bins.

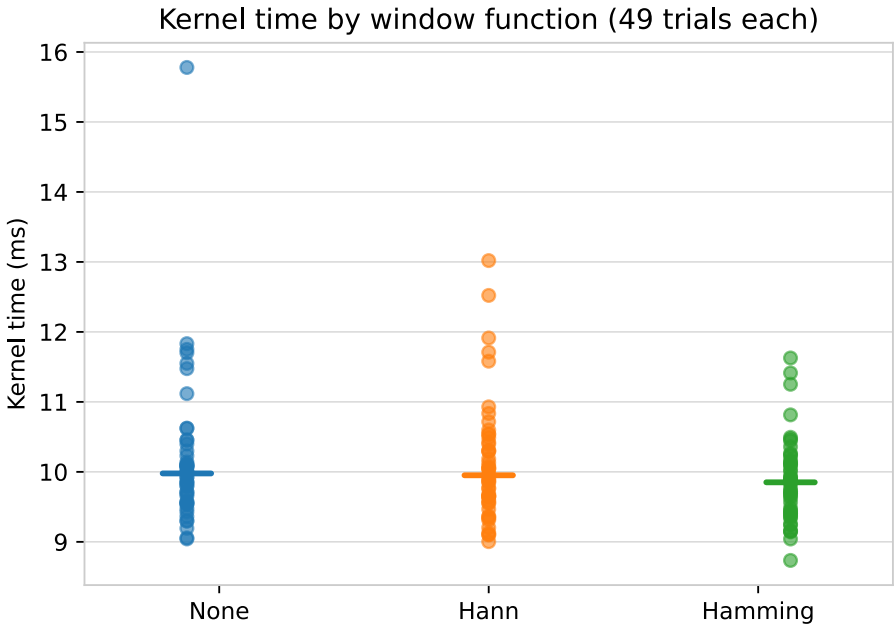



FIGURE 5.3: Latency contribution of the windowing stage to the overall pipeline. The latency does not significantly differ across implementations, with all implementations taking around 9-12 ms to process a single frame of 7 670 000 samples. The insignificant difference in latency between no filter and filters suggests that the delay is primarily due to the overhead of handling the data.

## 5.3 Coherent Integration

The coherent integration stage is validated on synthetic test data from software scripts and the test-source mode of the AWR2243 radar chips. The results from the former illustrate the expected increase in SNR with increasing integration window size as demonstrated in Figure 5.4. The results from the latter illustrate no increase in SNR because the test-source mode does not generate any noise. Meaningfully, the results from the AWR test-source mode indicate that the integration stage behaves correctly on data from the radar chips, (see Figure 5.5). The subsystem can process the data at a rate of 9.8 times the incoming data rate on the desktop setup, and 3.8 times the incoming data rate on the Xavier setup. We also find that the latency of the integration stage increases with the integration's factor of reduction, since in our implementation, each thread performs all the sum operations for its corresponding range bin (which increases with the reduction factor of the integration stage).

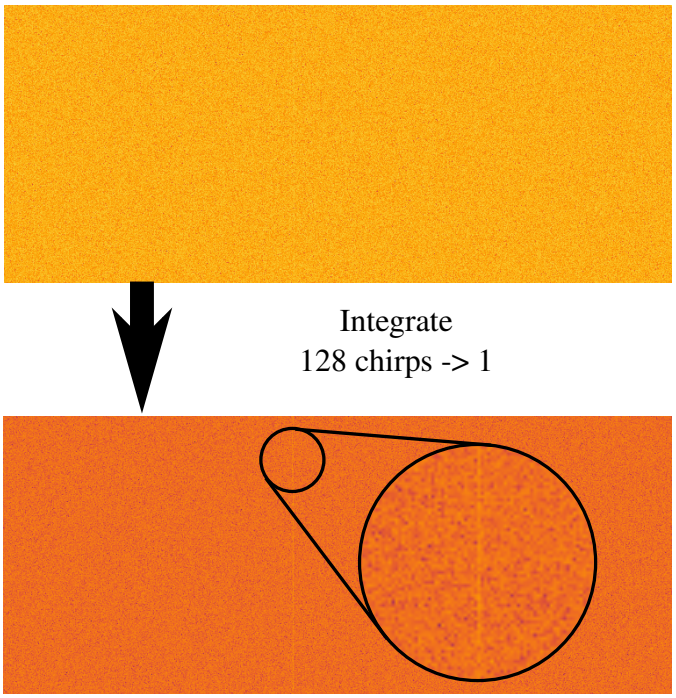



FIGURE 5.4: Results of the coherent integration stage on synthetic data. The top window shows a low SNR waterfall plot of data, with the y-axis representing range samples and the x-axis representing chirps that are streaming into the integration stage. The colour indicates the power of the range bin, with yellow being the most intense. The output of the integration stage below has less noise, and the target signal is more prominent.

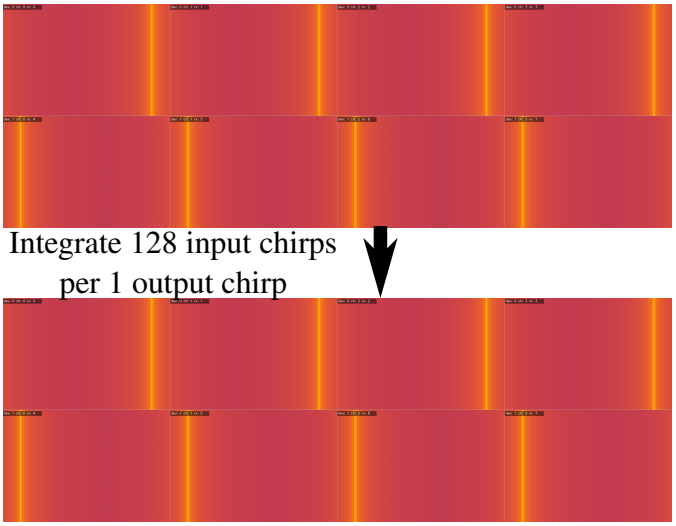



FIGURE 5.5: Results of the coherent integration stage on AWR2243 test-source data. The 8 windows correspond to 8 channels. 128 chirps are being compressed into a single chirp with no noise. There is no change in amplitude, validating that the integration stage behaves correctly on radar chip data.

## 5.4 Constant False Alarm Rate

### 5.4.1 Correctness

Under the assumption of an 8-degrees of freedom chi-squared distribution of quadruplet sample noise power, the CFAR threshold coefficient $\alpha$ is calculated numerically for a range of false alarm rates. The actual false alarm rate is then measured by running the CFAR algorithm on a noise-only synthetic dataset. Figure 5.6 illustrates the results.

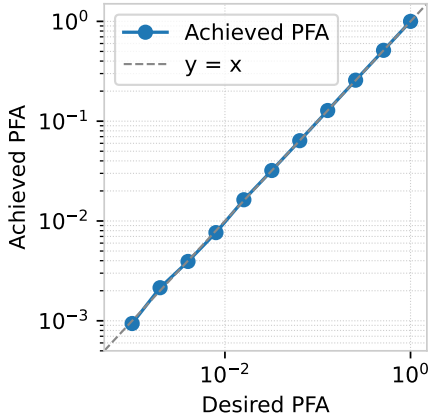



FIGURE 5.6: The actual false alarm rate closely matches the desired false alarm rate across a range of false alarm rates. The numerical solution for $\alpha$ is therefore accurate under the assumption of an 8-degrees of freedom chi-squared distribution of quadruplet sample noise power.

The results indicate that the numerical solution for $\alpha$ is accurate, with the actual false alarm rate closely matching the desired. The successful performance of CFAR with a false alarm rate of $0.001$ is illustrated in Figure 5.4.1.

### 5.4.2 Computational Performance

The speed of each of the three CFAR implementations is measured across a range of buffer sizes, from one frame of 767 samples, to 4042 frames of 767 samples. This sweep is performed on both the desktop and the Xavier. The results are illustrated in Figures 5.9 and 5.8.

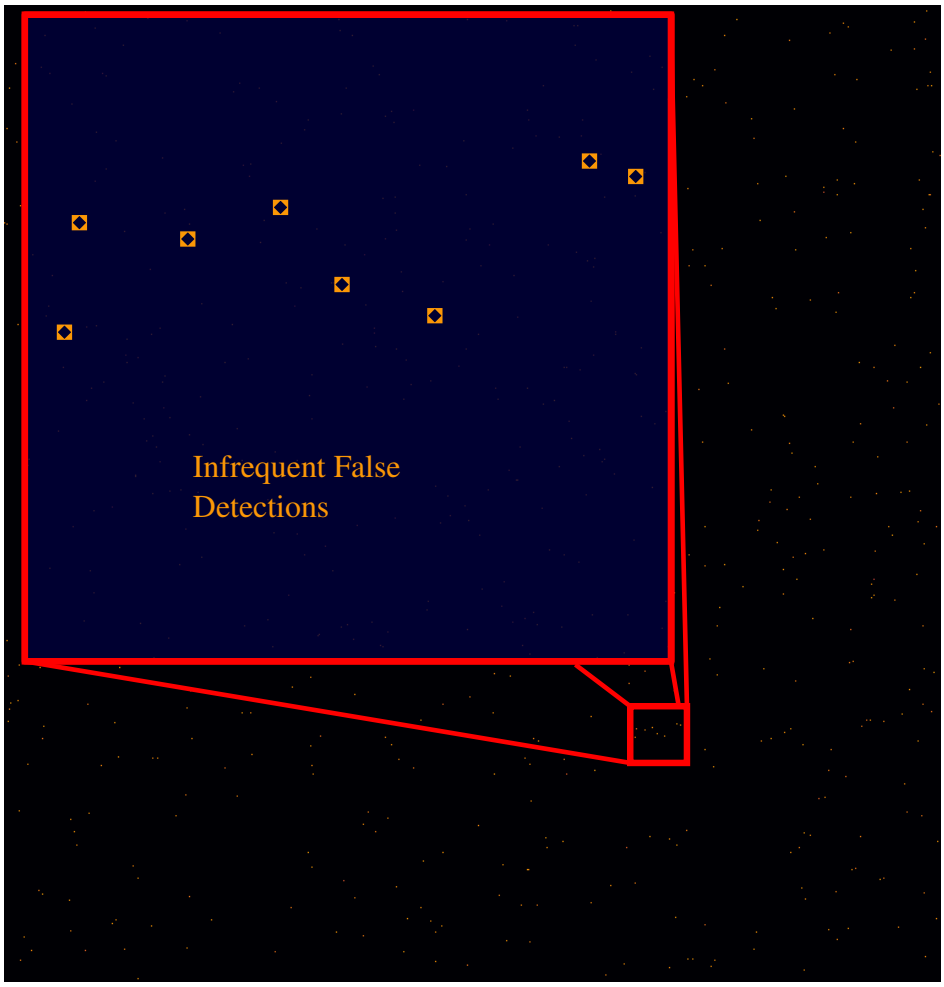



FIGURE 5.7: CFAR with a desired false alarm rate of $0.001$ infrequently detects peaks in what is predominantly noise, as expected. The figure is a waterfall plot, with chirp on the y-axis and range on the x-axis. The brightness of the cell reflects the power of the corresponding chirp's range bin. The black space indicates that most cells are masked by CFAR.

The speed is also plotted as a function of the number of CFAR training cells. The results are illustrated in Figures 5.11 and 5.10. The desktop results illustrate that both the GPU and the prefix sum table implementations are not significantly affected by the number of training cells, while the naive CPU implementation's speed increases linearly with the number of training cells. The Xavier results illustrate that while the prefix sum table implementation remains invariant to the number of training cells (as we expect), the GPU implementation's speed does increase with the number of training cells, due to the fact that each thread has more work to do to calculate the noise statistic. Results on the Xavier show that the GPU implementation

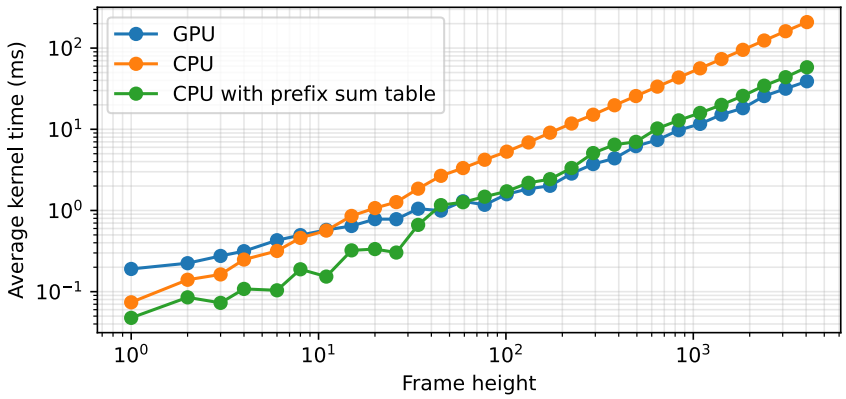



FIGURE 5.8: Computational Performance of three CFAR implementations as a function of buffer size on the desktop setup. The CPU implementation with a prefix sum tables is the fastest for small buffer sizes, but the GPU implementation is slightly faster for larger buffer sizes. The naive CPU implementation is the slowest across all buffer sizes.

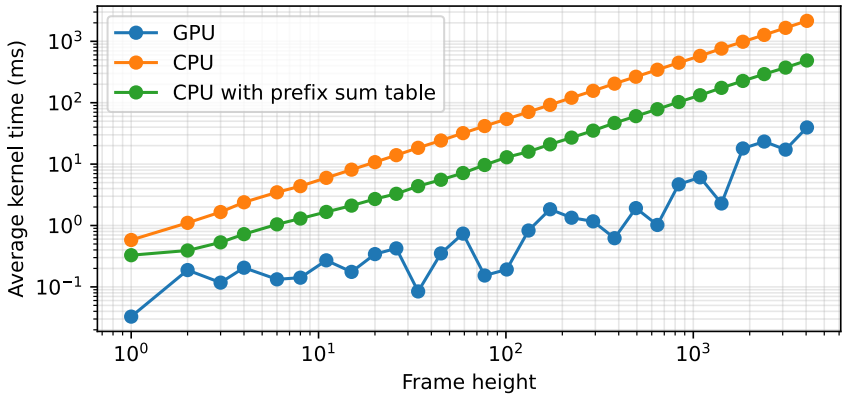



FIGURE 5.9: Computational Performance of three CFAR implementations as a function of buffer size on the Xavier setup. The CPU implementation with a prefix sum tables is the fastest for small buffer sizes, but the GPU implementation is slightly faster for larger buffer sizes. The naive CPU implementation is the slowest across all buffer sizes.

is faster than either CPU implementation for window sizes within the practical range of 10 - 100 training cells.

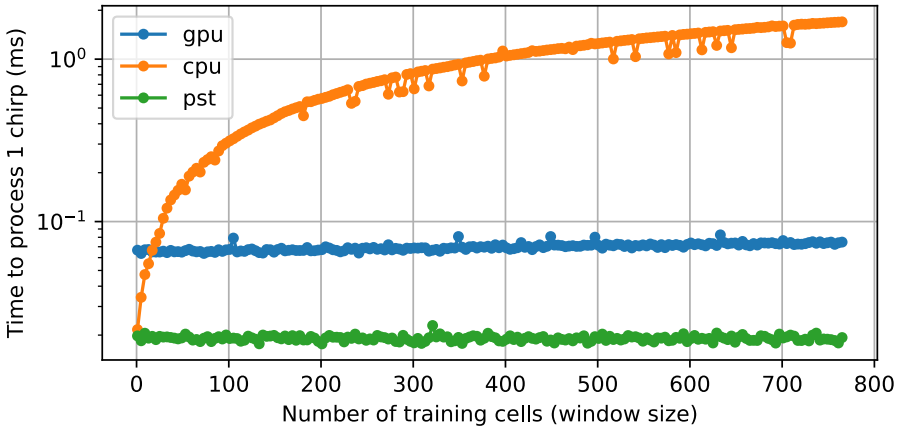



FIGURE 5.10: Computational Performance of three CFAR implementations as a function of the number of training cells on the Desktop Setup. Buffer size is fixed at 10 quadruplets.

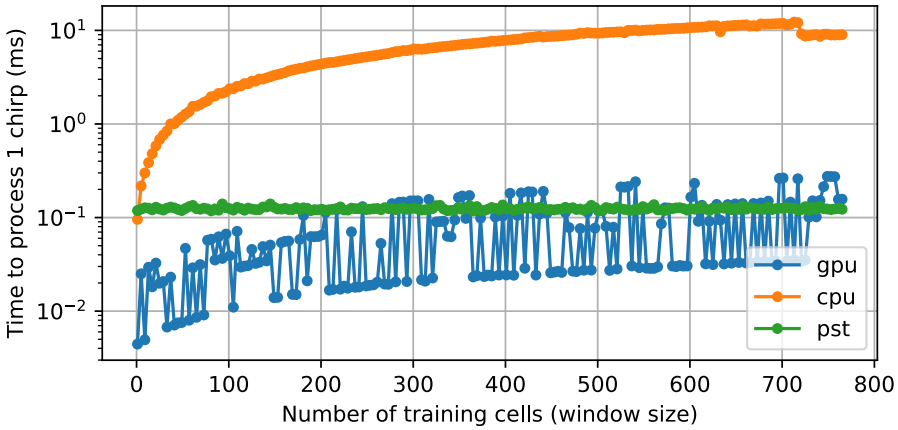



FIGURE 5.11: Computational Performance of three CFAR implementations as a function of the number of training cells on the Xavier Setup. Buffer size is fixed at 10 quadruplets.

### 5.4.3 Monopulse Angle Sensing

A simulation tool was built to validate the monopulse angle sensing algorithm on synthetic data. The tool allows the user to create multiple targets, and specify the bearing/elevation ratios of each target. The simulated data is then processed by the preprocessing subsystem, the aggregator, CFAR, and the monopulse angle sensing algorithm. The resulting angle estimates are visualised live in a 3D plot. A frame of this view is illustrated in Figure 5.12.

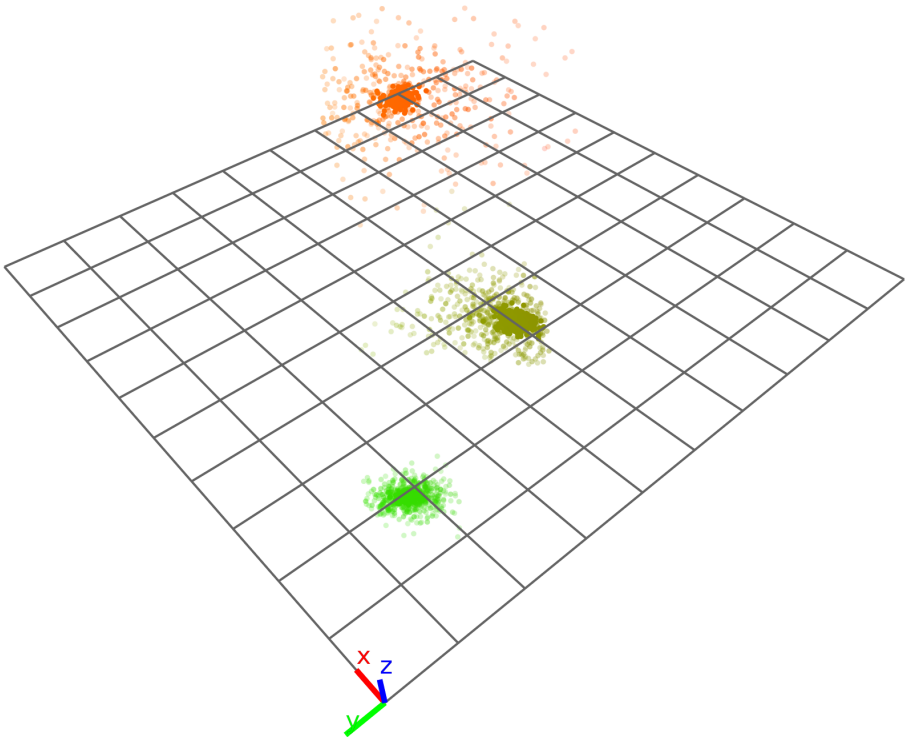



FIGURE 5.12: Results of the monopulse angle sensing algorithm on synthetic data. The colour of the dots indicate the range of the detections, with green being close and orange being far away. The opacity of the dots indicate the strength of the detections, with more opaque dots being stronger. The plot features the most recently 1000 detections. The spread of the dots increases with range as expected, and the upper and lower bounds of elevation and bearing match the field of view defined by the calibration polynomial.

The relative latency cost of the angle and CFAR operations were measured through an ablation study on the detection subsystem. The results are illustrated in Figure 5.13 which shows the latency of just CFAR, just angle sensing, and both, and Figure 5.14.

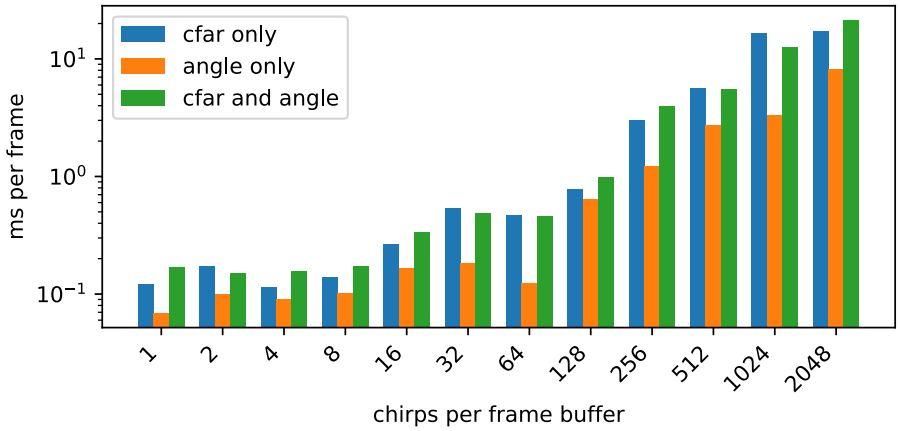



FIGURE 5.13: Latency of the subsystem running CFAR, angle sensing, and both on a log scale. The latency of the subsystem running CFAR is similar to the latency of the subsystem running both, while angle sensing is 1.5-5 times faster than either, suggesting that CFAR dominates the latency of the subsystem.

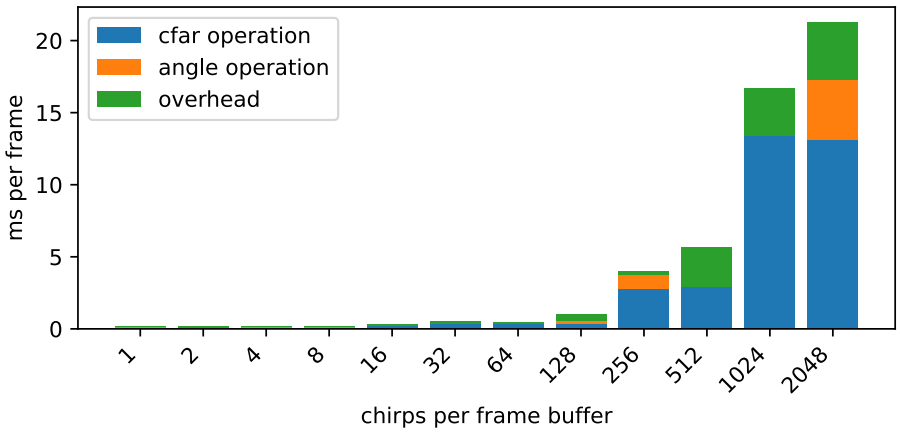



FIGURE 5.14: A breakdown of the latency of the subsystem estimated from the ablation study. Most of the latency is split between CFAR and the overhead of handling the data, which involves synchronising threads and migrating data between the CPU and GPU.

CHAPTER 6

# Discussion

In this chapter, we discuss the key findings within the results of the experiments. Then we discuss shortcomings of the research so far. Lastly, recommendations are given.

## 6.1 Findings

The results of the testing on the windowing function confirm the consensus in the literature, that the Hann and Hamming windows are both effective in reducing spectral leakage when compared to no window. Through comparing the two windows, we find that the Hamming window is more appropriate for radar purposes with its sharper drop-off that allows detection to be more precise, at the cost of more global spectral leakage. Additionally, the insignificant time difference between either window or no window is expected given the simplicity of the function, with each kernel performing just one multiplication. The negligible latency difference across the desktop and Xavier platforms further indicates that window selection can be made independently of hardware configuration. The differing spectral leakage patterns motivate a more comprehensive search for window functions that are most appropriate for miniature radar applications.

The results from the coherent integration experiment find that the signal to noise ratio improves by the square root of the number of chirps summed together under Gaussian noise, creating a positive downstream effect on the ability of CFAR to detect weak targets. The difference in throughput of the processing subsystem on the desktop versus the Xavier is likely due to the overhead of handling data, such as reading and writing. The large amount of work that each

thread has to do, summing over many chirps, motivates the exploration of more parallelised implementations that distribute the sums between threads.

The CFAR results outline a method for determining a threshold from a desired false alarm rate. The experiments demonstrate the GPU implementation is most appropriate for the Xavier embedded platform, regardless of the size of the buffer for all practical window sizes. The difference between hardware platforms highlights the importance of hardware-aware algorithm selection. The results of the angle sensing stage demonstrate the capability of the algorithm to detect multiple targets with different bearing/elevation ratios. The ablation study illustrates that CFAR dominates the latency of the tracking subsystem. The relatively high cost of CFAR motivates the exploration of more efficient CFAR implementations. Potential approaches include implementing the prefix sum table approach on the GPU, using a reduction approach to calculate the noise floor, and more consciously minimising cache misses within warps.

The completed results begin to address two of the three research gaps identified in the literature review. The GPU pipeline benchmarking across desktop and Xavier platforms, covering CFAR, angle sensing, windowing, and coherent integration, directly addresses gap 2, providing the hardware-validated latency, throughput, and architectural profiling that prior UAV DAA radar work lacked. The hardware-aware implementation comparisons partially address gap 3, establishing a baseline for angle sensing performance, though evaluation under realistic multi-target and non-Gaussian noise conditions remains to be completed. Gap 1, concerning detection sensitivity against small moving airborne targets, and the remaining scope of gap 3, require the tracking stage and real flight data, which are the subject of the remaining work.

## 6.2 Shortcomings

The project has progressed broadly in line with the initial proposal, with the preprocessing subsystem, CFAR detection, monopulse angle sensing, and the supporting simulation and profiling infrastructure all implemented and evaluated. The pipeline architecture has been

validated on both synthetic data and real AWR2243 hardware data, and hardware-aware implementation comparisons have been completed for the CFAR stage across the desktop and Xavier platforms.

Several goals from the initial proposal were not completed within this reporting period. The multi-target tracking stage, comprising clustering and the tracking filter, was not implemented or evaluated. This represents the most significant deviation from the proposed plan, and means that the end-to-end pipeline remains incomplete and that track quality cannot yet be used as an evaluation metric. The primary cause of this delay was the time required to bring up the hardware pipeline and validate each stage on real radar data. The remaining work in the project plan is structured to address this directly. Until the tracking stage is complete, gap 1 (detection sensitivity against small moving airborne targets) cannot be addressed, as it requires track-level performance metrics from real encounter scenarios.

The window function evaluation was narrowed to Hann and Hamming windows, deferring the evaluation of Blackman, Blackman–Harris, and Kaiser windows. The coherent integration comparison was limited to synthetic data for SNR validation and real test-source data for hardware correctness; the parallelised implementation comparison and downstream effect on angle sensing stability were not completed. The angle sensing evaluation did not include the proposed sweeps over SNR, channel imbalance, and multi-target proximity. These items are carried forward into the remaining work plan. Additionally, gap 3 — evaluating angle sensing algorithms under realistic noise conditions — remains open until real flight data is collected to characterise the noise regime of the AWR2243 in airborne operation.

## 6.3 Recommendations

The windowing results support the selection of the Hamming window for radar applications where resolution of closely spaced targets is a priority, given its steeper drop-off near the target bin relative to the Hann window. The global spectral leakage penalty of the Hamming window is acceptable in practice, as the noise floor is dominated by thermal and clutter contributions rather than windowing artefacts at operationally relevant SNRs. As the latency contribution

of the windowing stage is negligible and invariant to window choice, the selection can be made purely on detection performance grounds.

For the CFAR and angle sensing stages, the results support a GPU-based implementation on the Xavier embedded platform across all practical training cell window sizes. On the desktop, the prefix sum CPU implementation is competitive for small buffer sizes, but the GPU implementation is preferable at larger buffer sizes. The crossover point differs between platforms, confirming that implementation selection cannot be made platform-agnostically. As CFAR dominates the latency of the detection subsystem, with angle sensing contributing a factor of 1.5 to 5 times less, optimisation effort should be directed at the CFAR stage rather than the angle sensing stage.

The overhead of host–device data transfer and thread synchronisation constitutes a substantial portion of the remaining latency, and represents the primary target for further pipeline optimisation.

CHAPTER 7

# Review of Project Plan

---

## 7.1 Overview

This chapter provides an overview of the changes made to the original project plan, the methodology for the remaining work, the timeline for the remaining work, stakeholder management and risk management. Lastly, the contributions of the project to the learning objectives of the AMME5520 and ELEC3305 units are discussed.

## 7.2 Changes to proposed plan

The project plan remains broadly consistent with the original proposal. The core pipeline architecture, evaluation methodology, and hardware platforms are unchanged. However, several adjustments have been made in response to findings from the first phase of work, and some scope items have been deferred pending further information about the radar's noise regime in real operating conditions.

The most significant addition to the plan is a clustering stage, inserted between the angle sensing and tracking stages. This was not included in the original proposal but is motivated by the observed output of the angle sensing visualisation tool, which produces spatially distributed detections per target that must be grouped before being passed to the tracking filter. Without clustering, the data association stage would be presented with far more measurements per target than the tracking filter is designed to handle.

The evaluation of CFAR variants, CA, GO, SO, and OS-CFAR, and additional window functions such as Blackman, Blackman–Harris, and Kaiser, have been deferred rather than removed from scope. Both depend on a better characterisation of the radar's noise regime under real operating conditions, which was not available during the first phase. These evaluations will proceed once sufficient real data has been collected to determine which noise assumptions are appropriate.

Non-coherent integration has been removed from scope. Coherent integration is sufficient for the detection sensitivity requirements of the pipeline, and the additional complexity of summing chirps with different profiles constructively is not worth sacrificing research in the tracking subsystem.

The FFT comparison, PVA evaluation, pipeline design experiment, tracking filter comparison, data association comparison, and flight test validation remain in scope. The pipeline design experiment and CFAR/windowing extensions are sequenced after the core tracking pipeline is complete and unit tested, as meaningful pipeline-level optimisation requires a stable end-to-end system.

## 7.3 Methodology for Remaining Work

### 7.3.1 Clustering

Detections produced by the angle sensing stage are spatially distributed across multiple range-bearing-elevation cells per physical target. A clustering stage is required to group these detections into candidate target reports before data association. DBSCAN is the primary candidate, as it does not require a prior estimate of the number of clusters and is robust to the irregular cluster shapes expected from monopulse detections at varying range. The sensitivity of cluster quality to the epsilon and minimum points hyperparameters will be evaluated on synthetic multi-target data. CPU and GPU implementations will be compared for latency and throughput.

### 7.3.2 Tracking Filter

The tracking filter fuses clustered detections over time to produce continuous target tracks. Extended Kalman (EKF), unscented Kalman (UKF), and interacting multiple model (IMM) filters will be compared. Evaluation metrics include mean squared error (MSE), normalised estimation error squared (NEES), normalised innovation squared (NIS), average time-to-confirm, track fragmentation, and ID-switch rate under clutter, missed detections, and manoeuvres. CPU and GPU implementations will be compared for latency and throughput. The evaluation will determine whether the additional complexity of UKF and IMM over EKF produces measurable improvements in track quality that justify their computational cost.

### 7.3.3 Data Association

Nearest neighbour (NN), global nearest neighbour (GNN), and joint probabilistic data association (JPDA) will be compared under sweeps of target density and clutter rate. Performance will be assessed using CLEAR MOT metrics alongside latency and throughput measurements. The interaction between data association choice and tracking filter choice will be examined, as some combinations are known to degrade under high clutter.

### 7.3.4 FFT Comparison

CPU and GPU FFT implementations will be compared under streaming workloads using FFTW and MKL on CPU, and cuFFT on GPU. The NVIDIA Programmable Vision Accelerator (PVA) will also be evaluated as an alternative execution target for the FFT stage. Trade-offs will be characterised by latency, throughput, and memory usage at the FFT sizes required by the AWR2243 data format.

### 7.3.5 Pipeline Design

Once the end-to-end pipeline is stable and unit tested, pipeline-level implementation variants will be compared. Stream format choices, including interleaved versus per-channel layout

and float precision, will be evaluated alongside memory placement decisions, contrasting CPU memory, device memory, and unified memory configurations. Compatible stages will be evaluated for fusion opportunities. End-to-end latency and sustained throughput are the primary metrics. This directly targets the gap around memory movement and synchronisation overhead in embedded GPU pipelines.

### 7.3.6 CFAR Variants and Extended Windowing

The evaluation of GO, SO, and OS-CFAR variants, and the extended windowing comparison including Blackman, Blackman–Harris, and Kaiser windows, will proceed once sufficient real AWR2243 data has been collected to characterise the radar's noise regime. The appropriate CFAR variant and window function depend on whether the clutter and noise distributions encountered in real operation match the homogeneous Gaussian assumptions under which CA-CFAR and Hamming windowing were selected. These evaluations are therefore sequenced after initial field data collection.

### 7.3.7 Hardware Profiling

All experiments will be repeated across CPU-only, unified-memory CPU+GPU, and discrete-memory CPU+GPU configurations on the Xavier and desktop platforms. Power-constrained regimes will be emulated by restricting clock speeds, active cores, and power modes, then measuring degradation in latency, throughput, and track continuity. Results will identify which pipeline stages become bottlenecks as SWaP constraints tighten, and which algorithm and implementation choices remain viable under those conditions.

### 7.3.8 Flight Test Validation

The completed end-to-end pipeline will be validated on data collected from flight trials using the AWR2243 radar hardware. Flight test validation will confirm that detection and tracking performance observed in simulation and on synthetic data generalises to real airborne targets,

and will provide the operating noise regime data required for the deferred CFAR variant and windowing evaluations.

## 7.4 Timeline

The remaining project milestones are outlined in Table 7.1.

TABLE 7.1: Updated project milestones and deliverables (06 Apr to 13 Jul).

| Date range | Milestone |
|---|---|
| 20 Mar, 08 Apr | Finalise and submit progress report. |
| 06 Apr, 12 Apr | Implement clustering stage. Compare DBSCAN CPU and GPU implementations. |
| 13 Apr, 19 Apr | Implement and compare tracking filter variants (EKF, UKF, IMM) and data association variants (NN, GNN, JPDA) on synthetic data. Evaluate track quality metrics. |
| 20 Apr, 26 Apr | FFT comparison across FFTW, cuFFT, and PVA. Integration testing of complete end-to-end pipeline. |
| 26 Apr, 02 May | Field testing and flight trial validation on AWR2243 hardware. Collect real noise regime data. |
| 03 May, 09 May | Pipeline design experiment. Compare stream formats, memory placements, and stage fusion strategies. Hardware profiling across CPU-only, unified-memory, and discrete-memory configurations. |
| 10 May, 16 May | CFAR variant and extended windowing evaluation on real data. Sensitivity analysis and parameter optimisation. |
| 17 May, 15 Jun | Draft report featuring all simulation and real data experiment results, findings, and conclusions. |
| 15 Jun, 13 Jul | Final report featuring all real and simulated experiment results, findings, and conclusions. |

## 7.5 Stakeholder Management

There are three groups of stakeholders who need to be considered throughout the research: the industry partner Mission Systems, regulators such as CASA and EASA, and airspace users. The industry partner will be considered throughout the research through weekly meetings with the director and frequent consultation with the radar engineers, to ensure that the system being developed is useful and that the experiments being performed are insightful. In addition to contributing to the radar project directly, it is important that the tools, methods, and findings

are documented so that the partner can continue to make improvements after this research project concludes.

Regulators such as CASA and EASA have a stake in this research because DAA capability is a prerequisite for beyond visual line of sight (BVLOS) approval under the regulatory frameworks currently being developed. Evidence of achievable detection and tracking performance under SWaP constraints informs what performance requirements are technically feasible to mandate, and the benchmarking methodology developed in this project may provide a basis for standardised evaluation of DAA systems in future regulatory processes.

Airspace users comprise two groups with related but distinct interests. Manned aircraft operators have a safety interest in UAVs being able to reliably detect and avoid them in shared controlled airspace. Drone operators and commercial UAV service providers have a practical interest in DAA systems that are lightweight, power-efficient, and affordable enough to be deployable on small platforms. The SWaP-constrained design focus of this research directly addresses both concerns, and successful outcomes would support safer and more accessible integration of UAVs into controlled airspace.

## 7.6 Risk Management

The remaining work involves a combination of software development, hardware testing, and field trials, which present a range of risks from physical safety to data security. Table 7.2 outlines the key risks identified for the remaining work, along with their likelihood, severity, and mitigation strategies.

TABLE 7.2: Risk assessment for remaining project work.

| Risk | Likelihood | Severity | Mitigation |
|---|---|---|---|
| Drone collision with environment in trials | Low | High | Trials conducted in restricted and physically open airspace. Pre-flight checks completed before each test. |
| Fainting or heat exhaustion during field tests | Low | Medium | Field tests scheduled outside peak heat. Frequent water and rest breaks. Trials conducted with at least two people present. |
| Fire risk from LiPo battery | Low | High | Batteries stored and charged in fire-resistant bags. Damaged batteries disposed of appropriately. |
| Jetson Xavier NX overheating | Low | Low | Active heatsink fitted. Thermal throttling monitored during profiling runs. Experiments repeated if throttling is detected. |
| Musculoskeletal strain from prolonged sitting | Medium | Low | Regular breaks taken during extended development sessions. Frequent changes in posture. |
| Cybersecurity breach of research data or pipeline code | Low | Medium | Code stored in private repository with access controls. Raw radar data stored on internal network server. Limit exposure of online Large Language Models to proprietary software. |

## 7.7 Case Studies

### 7.7.1 AMME5520 Advanced Control and Optimisation

Progress towards the AMME5520 learning objectives has been partially achieved at this stage of the project. LO1, concerning critical engagement with research literature, has been fully addressed through the literature review and gap synthesis across UAV DAA radar pipelines, GPU radar processing, and multi-target tracking. LO4, concerning cost function framing and objective-driven trade-offs, has been partially addressed through the ablation study and parameter sweeps conducted for the CFAR and angle sensing stages, where detection sensitivity and computational cost are explicitly traded off. LO2 and LO3, which concern the implementation and evaluation of tracking filters, data association, and estimation algorithms, have not yet been addressed, as the tracking stage of the pipeline remains to be implemented.

These learning objectives will be met in the remaining work through the implementation and comparison of EKF, UKF, and IMM filters alongside NN, GNN, and JPDA data association variants.

### 7.7.2 ELEC3305 Digital Signal Processing

Progress towards the ELEC3305 learning objectives is more advanced at this stage. LO1, concerning the mathematical description of signals using transforms, convolutions, and spectral analysis, has been addressed through the implementation and evaluation of windowing, FFT, and coherent integration stages, which directly engage with frequency-domain radar processing and spectral leakage mitigation. LO2, concerning software implementation of DSP stages, has been substantially addressed through the development of the full preprocessing and detection subsystem in C++ and Python, including validation tooling, profiling scripts, and parameter sweeps. LO3, concerning linear signal models, has been partially addressed through the range-domain signal model used in the preprocessing and CFAR stages. LO4, concerning the design and evaluation of signal processing systems, has been partially addressed through the comparative evaluation of windowing functions and CFAR implementations; the remaining pipeline experiments will complete this coverage.

# Bibliography


[1] European Union Aviation Safety Agency (EASA). (Jul. 2024). Easy access rules for unmanned aircraft systems (regulations (eu) 2019/947 and 2019/945): Revision from july 2024. Online publication page (Revision from July 2024).

[2] RTCA, Inc. (Dec. 2025). Sc-228: Minimum operational performance standards for unmanned aircraft systems. Committee page; Terms of Reference listed as December 16, 2025.

[3] C. Serres, B. Gill, M. W. M. Edwards and M. G. Wu, 'Detect-and-avoid safety and operational suitability analysis using an electro-optical/infrared sensor model,' National Aeronautics and Space Administration, NASA Technical Memorandum NASA/TM–20210016872, Mar. 2021, Ames Research Center; MIT Lincoln Laboratory.

[4] B. Borden, *Radar Imaging of Airborne Targets: A Primer for Applied Mathematicians and Physicists*. CRC Press, 1999, Originally published 1999; later reprints available via CRC Press/Taylor & Francis.

[5] W. A. Lies, L. Narula, P. A. Iannucci and T. E. Humphreys, 'Low SWaP-C radar for urban air mobility,' in *Proceedings of the 2020 IEEE/ION Position, Location, and Navigation Symposium (PLANS)*, Preprint available at https://rnl.ae.utexas.edu/images/stories/files/papers/lies2020lowSwapc.pdf, Portland, OR, Apr. 2020.

[6] T. McLain, L. R. Sahawneh, M. O. Duffield and R. W. Beard, 'Detect and avoid for small unmanned aircraft systems using ADS-B,' *Air Traffic Control Quarterly*, vol. 23, no. 2–3, pp. 203–224, 2015, BYU ScholarsArchive preprint: https://scholarsarchive.byu.edu/facpub/1895.

[7] Z. Wang, X. Lin, X. Xiang, E. Blasch, K. Pham, G. Chen, D. Shen, B. Jia and G. Wang, 'An airborne low SWaP-C UAS sense and avoid system,' in *Proceedings of SPIE,*

*Sensors and Systems for Space Applications IX*, vol. 9838, Baltimore, MD, May 2016, p. 98380C.

[8] Echodyne Corp., *EchoFlight 4d airborne radar: Product datasheet*, Specifications: K-band 24.45–24.65 GHz; weight 817 g; 45 W operating power; size 18.7 cm × 12 cm × 4 cm. Retrieved 25 March 2026 from https://www.echodyne.com/media/ec0jv5tp/ts-echoflight-echodyne.pdf, 2024.

[9] Mordor Intelligence. (Jul. 2025). Airborne radars market size & analysis - growth trends & forecasts (2025 - 2030). Page last updated on July 24, 2025.

[10] K. Zaknoun, 'Real-time, gpu-accelerated processing of digital radar signal data,' Master's thesis, University of Turku, Jun. 2024.

[11] W. Feng, X. Hu and X. He, 'Artificial intelligence (ai)-based radar signal processing and radar imaging,' *Electronics*, vol. 13, no. 21, 2024.

[12] Civil Aviation Safety Authority, *New trial pathways for BVLOS approvals*, Web page, News item describing CASA's 12-month trial introducing new BVLOS approval pathways and expected supported use cases., Sep. 2025.

[13] Australian Government, Department of Infrastructure, Transport, Regional Development, Communications, Sport and the Arts, *Drones and bushfire capability*, Web page, Overview page describing the role of drones in bushfire scenarios and linking to the BRCoE RPAS Bushfire Roadmap.

[14] Australian Government, Department of Infrastructure, Transport, Regional Development, Communications and the Arts, 'Uncrewed aircraft systems (UAS) traffic management (UTM): A framework for UTM in australia and the actions we are taking,' Department of Infrastructure, Transport, Regional Development, Communications and the Arts, Action plan, Dec. 2024.

[15] RTCA, *DO-386: Minimum Operational Performance Standards for Airborne Collision Avoidance System Xu (ACAS Xu), Volumes I and II*, Prepared by Special Committee SC-147, Dec. 2020.

[16] EUROCAE, 'ED-275: Minimum Operational Performance Standard (MOPS) for ACAS Xu, Volumes I and II,' European Organisation for Civil Aviation Equipment, Tech. Rep., Dec. 2020, Prepared by Working Group WG-75 SG-1.

[17] ICAO, ‘Annex 10 – Aeronautical Telecommunications, Volume IV – Surveillance Radar and Collision Avoidance Systems, Amendment 91,’ International Civil Aviation Organization, Tech. Rep., 2022, Applicable 3 November 2022. Introduces ACAS Xa/Xo provisions.

[18] Federal Aviation Administration, ‘Normalizing Unmanned Aircraft Systems Beyond Visual Line of Sight Operations,’ Federal Aviation Administration and Transportation Security Administration, Tech. Rep., Aug. 2025, Notice of Proposed Rulemaking, 90 FR 38212, Docket No. FAA-2025-1908, Notice No. 25-07.

[19] Civil Aviation Safety Authority, *Temporary Management Instruction 2025-03: Broad Area BVLOS Operations*, Requires DAA mitigations for cooperative and non-cooperative air risk, Oct. 2025.

[20] ASTM International, ‘F3442/F3442M-23: Standard Specification for Detect and Avoid System Performance Requirements,’ ASTM International, Tech. Rep., Feb. 2023, Committee F38 on Unmanned Aircraft Systems.

[21] RTCA, ‘DO-365C: Minimum Operational Performance Standards (MOPS) for Detect and Avoid (DAA) Systems,’ RTCA, Inc., Tech. Rep., Sep. 2022, Prepared by Special Committee SC-228.

[22] EUROCAE, ‘ED-322: System Performance and Interoperability Requirements for Non-Cooperative UAS Detection Systems,’ European Organisation for Civil Aviation Equipment, Tech. Rep., 2022, References DO-365C for cooperative/non-cooperative distinctions.

[23] M. I. Skolnik, *Introduction to Radar Systems*, 3rd ed. McGraw-Hill, 2001.

[24] M. A. Richards, *Fundamentals of Radar Signal Processing*. McGraw-Hill Professional, 2005.

[25] M. A. Richards, J. A. Scheer and W. A. Holm, *Principles of Modern Radar: Basic Principles*. Raleigh, NC: SciTech Publishing, 2010, ch. 6.

[26] S. M. Sherman and D. K. Barton, *Monopulse Principles and Techniques*, 2nd ed. Norwood, MA: Artech House, 2011.

[27] A. H. Jazwinski, *Stochastic Processes and Filtering Theory*. Academic Press, 1970.

[28] H. A. P. Blom and Y. Bar-Shalom, 'The interacting multiple model algorithm for systems with markovian switching coefficients,' *IEEE Transactions on Automatic Control*, vol. 33, no. 8, pp. 780–783, 1988.

[29] S. J. Julier and J. K. Uhlmann, 'New extension of the kalman filter to nonlinear systems,' in *Signal Processing, Sensor Fusion, and Target Recognition VI*, ser. Proceedings of SPIE, vol. 3068, 1997, pp. 182–193.

[30] Y. Bar-Shalom and T. E. Fortmann, *Tracking and Data Association*. Orlando, FL: Academic Press, 1988.

[31] D. B. Reid, 'An algorithm for tracking multiple targets,' *IEEE Transactions on Automatic Control*, vol. 24, no. 6, pp. 843–854, 1979.

[32] S. S. Blackman and R. Popoli, *Design and Analysis of Modern Tracking Systems*. Norwood, MA: Artech House, 1999.

[33] NVIDIA Corporation, *CUDA programming guide, v13.1*, Last updated December 12, 2025, NVIDIA Corporation, 2025.

[34] D. Sysak, A. Byndas, T. Karas and G. Jaromi, 'Lightweight and compact pulse radar for uav platforms for mid-air collision avoidance,' *Sensors (Basel)*, vol. 25, no. 23, p. 7392, Dec. 2025.

[35] A. Moses, M. J. Rutherford and K. P. Valavanis, 'Radar-based detection and identification for miniature air vehicles,' in *2011 IEEE International Conference on Control Applications (CCA)*, 2011, pp. 933–940.

[36] L. Shi, C. Allen, M. Ewing, S. Keshmiri, M. Zakharov, F. Florencio, N. Niakan and R. Knight, 'Multichannel sense-and-avoid radar for small uavs,' in *2013 IEEE/AIAA 32nd Digital Avionics Systems Conference (DASC)*, 2013, 6E2-1-6E2–10.

[37] L. Newmeyer, D. Wilde, B. Nelson and M. Wirthlin, 'Efficient processing of phased array radar in sense and avoid application using heterogeneous computing,' in *2016 26th International Conference on Field Programmable Logic and Applications (FPL)*, 2016, pp. 1–8.

[38] X. Zhao, P. Liu, B. Wang and Y. Jin, 'Gpu-accelerated signal processing for passive bistatic radar,' *Remote Sensing*, vol. 15, no. 22, 2023.

[39] K. Zaknoun, 'Real-time, gpu-accelerated processing of digital radar signal data,' Master of Science in Technology Thesis, University of Turku, Turku, Finland, 2024.

[40] Q. An, C. Yeh, Y. Lu, Y. He and J. Yang, 'Time-varying angle estimation of multiple unresolved extended targets via monopulse radar,' *IEEE Transactions on Aerospace and Electronic Systems*, vol. 60, no. 4, pp. 4650–4667, 2024.

[41] A. Aubry, A. D. Maio, S. Marano and M. Rosamilia, 'Single-pulse simultaneous target detection and angle estimation in a multichannel phased array radar,' *IEEE Transactions on Signal Processing*, vol. 68, pp. 6649–6664, 2020.

[42] M. Ezuma, O. Ozdemir, C. K. Anjinappa, W. A. G. Khawaja and I. Guvenc, 'Micro-UAV detection with a low-grazing angle millimeter wave radar,' in *2019 IEEE Radio and Wireless Symposium (RWS)*, 2019, pp. 1–4.